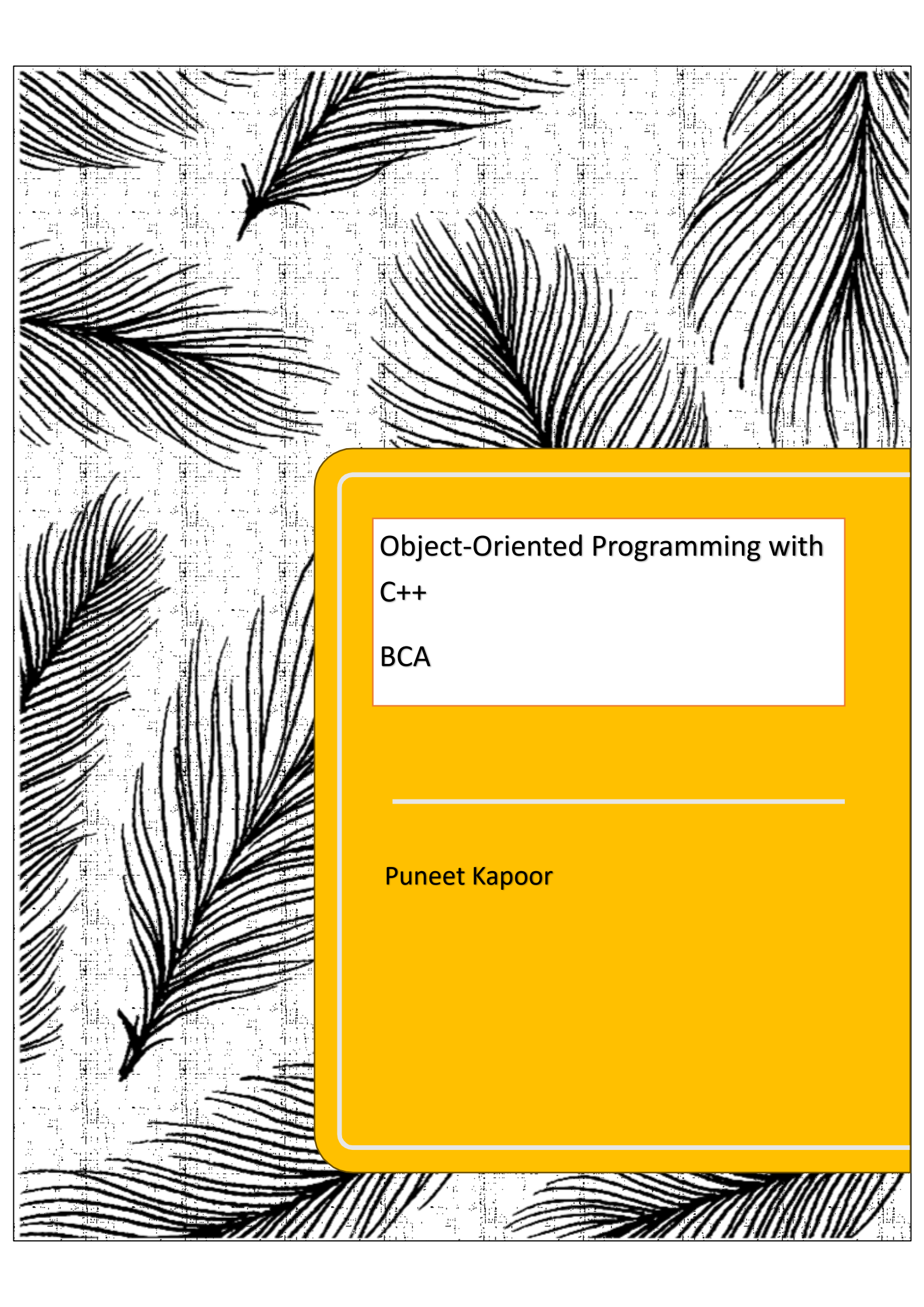




# Object-Oriented Programming with C++

BCA

---

Puneet Kapoor

## INDEX

|                                                        |           |
|--------------------------------------------------------|-----------|
| <b>Chapter 1: Getting Started with C++ .....</b>       | <b>8</b>  |
| 1.1 Objectives.....                                    | 8         |
| 1.2 Introduction to C++ .....                          | 8         |
| 1.2.1 What is C++? .....                               | 8         |
| 1.2.2 Evolution of C++ (Versions Over the Years).....  | 8         |
| 1.2.3 Key Features of C++ .....                        | 8         |
| 1.3 Setting Up C++ Programming Environment.....        | 8         |
| 1.3.1 Popular C++ Compilers.....                       | 9         |
| 1.3.2 Installing a C++ Compiler (GCC) .....            | 9         |
| 1.3.3 Choosing an IDE for C++ Development .....        | 9         |
| 1.4 Writing and Running a Simple C++ Program.....      | 9         |
| 1.4.1 Basic Structure of a C++ Program.....            | 9         |
| 1.4.2 Explanation of Code .....                        | 9         |
| 1.4.3 Compiling and Running the Program .....          | 9         |
| 1.5 Summary .....                                      | 10        |
| 1.6 Keywords .....                                     | 10        |
| 1.7 Review Questions.....                              | 10        |
| 1.8 Further Reading.....                               | 10        |
| <b>Chapter 2: Fundamentals of C++ Programming.....</b> | <b>11</b> |
| 2.1 Objectives.....                                    | 11        |
| 2.2 Variables and Data Types .....                     | 11        |
| 2.2.1 What is a Variable?.....                         | 11        |
| 2.2.2 Data Types in C++.....                           | 11        |
| 2.3 Operators in C++ .....                             | 12        |
| 2.3.1 Arithmetic Operators .....                       | 12        |
| 2.3.2 Comparison Operators.....                        | 12        |
| 2.4 Control Statements in C++.....                     | 13        |
| 2.4.1 If-Else Statement .....                          | 13        |
| 2.4.2 Loops in C++ .....                               | 13        |
| 2.5 Summary .....                                      | 13        |
| 2.6 Keywords .....                                     | 14        |
| 2.7 Multiple-Choice Questions (MCQs) .....             | 14        |
| □ Keywords.....                                        | 14        |
| □ Further Reading .....                                | 15        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| <b>Chapter 3: Functions and Memory Management in C++ .....</b> | <b>15</b> |
| 3.1 Objectives.....                                            | 15        |
| 3.2 Functions in C++ .....                                     | 15        |
| 3.2.1 What is a Function? .....                                | 15        |
| 3.2.2 Function Declaration, Definition, and Calling .....      | 15        |
| 3.3 Default Arguments in Functions .....                       | 16        |
| 3.4 Memory Management in C++ .....                             | 17        |
| 3.5.1 Dynamic Memory Allocation (new and delete).....          | 17        |
| 3.5 Summary .....                                              | 17        |
| 3.6 Keywords .....                                             | 17        |
| 3.7 Multiple-Choice Questions (MCQs) .....                     | 17        |
| 3.9 Further Reading.....                                       | 18        |
| <b>Chapter 4: Classes and Objects in C++ .....</b>             | <b>18</b> |
| 4.1 Objectives.....                                            | 18        |
| 4.2 Introduction to Object-Oriented Programming (OOP) .....    | 19        |
| 4.2.1 Key Concepts of OOP .....                                | 19        |
| 4.3 Understanding Classes and Objects .....                    | 19        |
| 4.3.1 What is a Class?.....                                    | 19        |
| 4.3.2 What is an Object?.....                                  | 19        |
| 4.4 Defining and Using Classes in C++ .....                    | 19        |
| 4.4.1 Defining a Class.....                                    | 19        |
| 4.4.2 Creating Objects from a Class .....                      | 20        |
| 4.5 Accessing Class Members.....                               | 20        |
| Example: Accessing Class Members.....                          | 20        |
| 4.6 Memory Allocation of Objects in C++ .....                  | 21        |
| Example: Multiple Objects and Independent Memory.....          | 21        |
| 4.7 Summary .....                                              | 22        |
| 4.8 Keywords .....                                             | 22        |
| 4.9 Multiple-Choice Questions (MCQs) .....                     | 22        |
| 4.10 Further Reading.....                                      | 23        |
| <b>Chapter 5: Constructors and Destructors in C++ .....</b>    | <b>23</b> |
| 5.1 Objectives.....                                            | 23        |
| 5.2 Introduction to Constructors and Destructors .....         | 23        |
| 5.2.1 What is a Constructor? .....                             | 23        |
| 5.3 Types of Constructors .....                                | 24        |
| 5.3.1 Default Constructor .....                                | 24        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| 5.3.2 Parameterized Constructor .....                                         | 25        |
| 5.3.3 Copy Constructor.....                                                   | 25        |
| 5.4 What is a Destructor? .....                                               | 26        |
| 5.5 Constructor vs Destructor: Key Differences .....                          | 27        |
| 5.6 Summary .....                                                             | 27        |
| 5.7 Keywords .....                                                            | 27        |
| 5.8 Multiple-Choice Questions (MCQs) .....                                    | 27        |
| 5.9 Further Reading.....                                                      | 28        |
| <b>Chapter 6: Inheritance in C++ .....</b>                                    | <b>28</b> |
| 6.1 Objectives.....                                                           | 28        |
| 6.2 What is Inheritance?.....                                                 | 28        |
| 6.2.1 Why Use Inheritance? .....                                              | 29        |
| 6.3 Types of Inheritance in C++ .....                                         | 29        |
| 6.4 Single Inheritance.....                                                   | 29        |
| 6.4.1 Understanding Base and Derived Classes .....                            | 29        |
| 6.5 Multiple Inheritance .....                                                | 30        |
| 6.6 Multilevel Inheritance .....                                              | 30        |
| 6.7 Function Overriding in Inheritance .....                                  | 31        |
| 6.8 Summary .....                                                             | 32        |
| 6.9 Keywords .....                                                            | 32        |
| 6.10 Multiple-Choice Questions (MCQs) .....                                   | 32        |
| 6.11 Further Reading.....                                                     | 33        |
| <b>Chapter 7: Multiple and Multilevel Inheritance in C++ .....</b>            | <b>33</b> |
| 7.1 Objectives.....                                                           | 33        |
| 7.2 Multiple Inheritance in C++ .....                                         | 34        |
| 7.2.1 What is Multiple Inheritance? .....                                     | 34        |
| 7.2.2 Example: Multiple Inheritance .....                                     | 34        |
| 7.3 The Diamond Problem in Multiple Inheritance .....                         | 35        |
| 7.3.1 What is the Diamond Problem?.....                                       | 35        |
| 7.3.2 Solving the Diamond Problem Using Virtual Inheritance.....              | 35        |
| 7.4 Multilevel Inheritance in C++ .....                                       | 36        |
| 7.4.1 What is Multilevel Inheritance? .....                                   | 36        |
| 7.4.2 Example: Multilevel Inheritance .....                                   | 36        |
| 7.5 Advantages and Disadvantages of Multiple and Multilevel Inheritance ..... | 37        |
| 7.6 Summary .....                                                             | 37        |
| 7.7 Keywords .....                                                            | 37        |
| 7.8 Multiple-Choice Questions (MCQs) .....                                    | 37        |

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 7.9 Further Reading.....                                          | 38        |
| <b>Chapter 8: Polymorphism in C++ .....</b>                       | <b>38</b> |
| 8.1 Objectives.....                                               | 38        |
| 8.2 What is Polymorphism? .....                                   | 39        |
| 8.3 Types of Polymorphism in C++ .....                            | 39        |
| 8.4 Compile-Time (Static) Polymorphism .....                      | 39        |
| 8.4.1 Function Overloading .....                                  | 39        |
| 8.4.2 Operator Overloading .....                                  | 40        |
| 8.5 Run-Time (Dynamic) Polymorphism.....                          | 41        |
| 8.5.1 Function Overriding .....                                   | 41        |
| 8.5.2 Virtual Functions .....                                     | 41        |
| 8.6 Summary .....                                                 | 42        |
| 8.7 Keywords .....                                                | 42        |
| 8.8 Multiple-Choice Questions (MCQs) .....                        | 42        |
| 8.9 Further Reading.....                                          | 43        |
| <b>Chapter 9: Applications of Polymorphism in C++ .....</b>       | <b>44</b> |
| 9.1 Objectives.....                                               | 44        |
| 9.2 Introduction to Applications of Polymorphism.....             | 44        |
| 9.3 Application of Function Overloading .....                     | 44        |
| 9.3.1 Function Overloading in Mathematical Computation .....      | 44        |
| 9.4 Application of Operator Overloading .....                     | 45        |
| 9.5 Application of Function Overriding.....                       | 46        |
| 9.6 Application of Virtual Functions in Real-World Software ..... | 46        |
| Example: Virtual Functions in a Database System .....             | 47        |
| 9.7 Summary .....                                                 | 47        |
| 9.8 Keywords .....                                                | 48        |
| 9.9 Multiple-Choice Questions (MCQs) .....                        | 48        |
| 9.10 Further Reading.....                                         | 48        |
| <b>Chapter 10: Abstract Classes and Interfaces in C++ .....</b>   | <b>49</b> |
| 10.1 Objectives.....                                              | 49        |
| 10.2 Introduction to Abstract Classes .....                       | 49        |
| 10.2.1 Why Use Abstract Classes? .....                            | 49        |
| 10.3 Pure Virtual Functions .....                                 | 49        |
| 10.4 Abstract Classes vs. Interfaces in C++ .....                 | 50        |
| 10.5 Real-World Applications of Abstract Classes.....             | 50        |
| 10.6 Example: Abstract Class in a Banking System .....            | 51        |
| 10.7 Summary .....                                                | 51        |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 10.8 Keywords .....                                                   | 52        |
| 10.9 Multiple-Choice Questions (MCQs) .....                           | 52        |
| 10.10 Further Reading.....                                            | 52        |
| <b>Chapter 11: Exception Handling in C++ .....</b>                    | <b>53</b> |
| 11.1 Objectives.....                                                  | 53        |
| 11.2 Introduction to Exception Handling.....                          | 53        |
| 11.3 The Try-Catch Mechanism in C++ .....                             | 53        |
| 11.3.1 Syntax of Exception Handling.....                              | 53        |
| 11.4 Throwing and Catching Exceptions .....                           | 54        |
| 11.4.1 Throwing an Exception .....                                    | 54        |
| 11.5 Catching Multiple Exceptions.....                                | 54        |
| 11.6 Using Standard Exception Classes in C++ .....                    | 55        |
| Example: Using Standard Exceptions .....                              | 55        |
| 11.7 Handling All Exceptions Using <code>catch ( . . . )</code> ..... | 56        |
| 11.8 Summary .....                                                    | 56        |
| 11.9 Keywords .....                                                   | 56        |
| 11.10 Multiple-Choice Questions (MCQs) .....                          | 57        |
| 11.11 Further Reading.....                                            | 57        |
| <b>Chapter 12: Standard Template Library (STL) in C++ .....</b>       | <b>57</b> |
| 12.1 Objectives.....                                                  | 57        |
| 12.2 Introduction to STL .....                                        | 58        |
| 12.2.1 Why Use STL? .....                                             | 58        |
| 12.3 STL Components.....                                              | 58        |
| 12.4 STL Containers .....                                             | 58        |
| 12.4.1 Vectors: Dynamic Arrays in C++ .....                           | 58        |
| 12.4.2 Maps: Key-Value Pair Storage .....                             | 59        |
| 12.5 STL Algorithms.....                                              | 60        |
| 12.5.1 Sorting Elements in a Vector .....                             | 60        |
| 12.6 STL Iterators .....                                              | 60        |
| 12.7 Summary .....                                                    | 61        |
| 12.8 Keywords .....                                                   | 61        |
| 12.9 Multiple-Choice Questions (MCQs) .....                           | 61        |
| 12.10 Further Reading.....                                            | 62        |
| <b>Chapter 13: Smart Pointers in C++ .....</b>                        | <b>62</b> |
| 13.1 Objectives.....                                                  | 62        |
| 13.2 Introduction to Smart Pointers .....                             | 62        |

|                                                                |           |
|----------------------------------------------------------------|-----------|
| 13.3 Why Use Smart Pointers? .....                             | 63        |
| 13.4 Types of Smart Pointers in C++ .....                      | 63        |
| 1 □ <code>unique_ptr</code> – Exclusive Ownership.....         | 63        |
| 2 □ <code>shared_ptr</code> – Shared Ownership .....           | 64        |
| 3 □ <code>weak_ptr</code> – Avoiding Circular References ..... | 64        |
| 13.5 Choosing the Right Smart Pointer.....                     | 65        |
| 13.6 Summary .....                                             | 65        |
| 13.7 Keywords .....                                            | 65        |
| 13.8 Multiple-Choice Questions (MCQs) .....                    | 65        |
| 13.9 Further Reading.....                                      | 66        |
| <b>Chapter 14: Multithreading and Concurrency in C++ .....</b> | <b>66</b> |
| 14.1 Objectives.....                                           | 66        |
| 14.2 Introduction to Multithreading .....                      | 66        |
| 14.3 Creating Threads in C++ .....                             | 67        |
| 14.4 Joining and Detaching Threads .....                       | 67        |
| 14.5 Race Conditions and Synchronization .....                 | 68        |
| 14.5.1 Using Mutex for Synchronization .....                   | 68        |
| 14.5.2 Using Lock Guard for Safer Mutex Handling .....         | 69        |
| 14.6 Condition Variables for Thread Communication.....         | 69        |
| 14.7 Summary .....                                             | 70        |
| 14.8 Keywords .....                                            | 70        |
| 14.9 Multiple-Choice Questions (MCQs) .....                    | 70        |
| 14.10 Further Reading.....                                     | 71        |
| <b>Chapter 15: File Handling in C++ .....</b>                  | <b>71</b> |
| 15.1 Objectives.....                                           | 71        |
| 15.2 Introduction to File Handling.....                        | 71        |
| 15.3 Opening and Closing Files .....                           | 72        |
| 15.3.1 Opening a File .....                                    | 72        |
| 15.4 Writing to a File .....                                   | 72        |
| 15.5 Reading from a File.....                                  | 73        |
| 15.6 Appending to a File .....                                 | 73        |
| 15.7 Working with Binary Files .....                           | 74        |
| 15.8 Checking File Status (Error Handling).....                | 75        |
| 15.9 Summary .....                                             | 76        |
| 15.10 Keywords .....                                           | 76        |
| 15.11 Multiple-Choice Questions (MCQs) .....                   | 76        |

|                                                                 |           |
|-----------------------------------------------------------------|-----------|
| 15.12 Further Reading.....                                      | 77        |
| <b>Chapter 16: Best Practices and Optimization in C++ .....</b> | <b>77</b> |
| 16.1 Objectives.....                                            | 77        |
| 16.2 Writing Clean and Readable Code .....                      | 77        |
| 16.3 Memory Management Best Practices.....                      | 78        |
| 16.4 Optimizing Loops for Performance.....                      | 78        |
| 16.5 Using Efficient Data Structures.....                       | 79        |
| 16.6 Reducing Function Overhead with Inline Functions.....      | 79        |
| 16.7 Using Move Semantics for Performance .....                 | 80        |
| 16.8 Summary .....                                              | 80        |
| 16.9 Keywords .....                                             | 80        |
| 16.10 Multiple-Choice Questions (MCQs) .....                    | 81        |
| 16.11 Further Reading.....                                      | 81        |



# Chapter 1: Getting Started with C++

## 1.1 Objectives

By the end of this chapter, you will be able to:

- Understand the **history and evolution of C++**.
- Learn about the **features and advantages of C++**.
- Set up a **C++ programming environment**.
- Write and execute a **simple C++ program**.

---

## 1.2 Introduction to C++

### 1.2.1 What is C++?

C++ is a **general-purpose programming language** developed by **Bjarne Stroustrup** in **1979** at **Bell Labs**. It was designed as an **extension of the C language**, incorporating **object-oriented programming (OOP) features** while maintaining **high efficiency and performance**.

C++ is widely used in:

- ✓ **Software Development** (Operating Systems, Browsers, Compilers)
- ✓ **Game Development** (Unreal Engine, Unity)
- ✓ **Embedded Systems** (Robotics, IoT Devices)
- ✓ **High-Performance Computing** (Scientific Computing, AI, Machine Learning)

### 1.2.2 Evolution of C++ (Versions Over the Years)

| C++ Version | Year | Key Features Introduced                                              |
|-------------|------|----------------------------------------------------------------------|
| C++98       | 1998 | Standard Template Library (STL), Exception Handling                  |
| C++03       | 2003 | Minor improvements & bug fixes                                       |
| C++11       | 2011 | Auto keyword, Smart Pointers, Lambda Functions                       |
| C++14       | 2014 | Performance improvements, Better Type Inference                      |
| C++17       | 2017 | <code>if constexpr</code> , Parallel Algorithms, File System Library |
| C++20       | 2020 | Concepts, Coroutines, Ranges, Modules                                |

### 1.2.3 Key Features of C++

- ✓ **Multi-Paradigm** – Supports Procedural, Object-Oriented, and Generic Programming.
- ✓ **Performance-Oriented** – Allows low-level memory management like C.
- ✓ **Object-Oriented** – Includes **Encapsulation, Inheritance, and Polymorphism**.
- ✓ **Portable & Fast** – Runs efficiently across different platforms.
- ✓ **Standard Template Library (STL)** – Built-in **Data Structures & Algorithms**.

---

## 1.3 Setting Up C++ Programming Environment

To write and run C++ programs, you need:

1. **A C++ Compiler** – Translates your C++ code into machine code.
2. **An Integrated Development Environment (IDE)** – Provides tools for writing, compiling, and debugging programs.

### 1.3.1 Popular C++ Compilers

| Compiler                                    | Platform              | Usage                    |
|---------------------------------------------|-----------------------|--------------------------|
| <b>GCC (GNU Compiler Collection)</b>        | Windows, Linux, macOS | Open-source, widely used |
| <b>Clang</b>                                | Windows, Linux, macOS | Fast and modern compiler |
| <b>MSVC (Microsoft Visual C++ Compiler)</b> | Windows               | Used with Visual Studio  |

### 1.3.2 Installing a C++ Compiler (GCC)

- ✓ **Windows:** Install **MinGW** (Minimalist GNU for Windows).
- ✓ **Linux/macOS:** Run `sudo apt install g++` (Ubuntu) or `brew install gcc` (Mac).

### 1.3.3 Choosing an IDE for C++ Development

| IDE                       | Features                       |
|---------------------------|--------------------------------|
| <b>Code::Blocks</b>       | Lightweight, beginner-friendly |
| <b>Dev-C++</b>            | Simple and fast for Windows    |
| <b>Visual Studio Code</b> | Extensions for C++ development |
| <b>CLion</b>              | Powerful and feature-rich      |

## 1.4 Writing and Running a Simple C++ Program

### 1.4.1 Basic Structure of a C++ Program

```
#include <iostream> // Standard Input/Output Library
using namespace std;

int main() {
    cout << "Hello, World!"; // Output statement
    return 0;
}
```

### 1.4.2 Explanation of Code

| Code Part                                   | Explanation                                                                             |
|---------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>#include &lt;iostream&gt;</code>      | Includes the input-output library (required for <code>cout</code> & <code>cin</code> ). |
| <code>using namespace std;</code>           | Allows using <code>cout</code> without <code>std::</code> prefix.                       |
| <code>int main()</code>                     | The <b>main function</b> – where execution begins.                                      |
| <code>cout &lt;&lt; "Hello, World!";</code> | Prints text to the screen.                                                              |
| <code>return 0;</code>                      | Indicates <b>successful program execution</b> .                                         |

### 1.4.3 Compiling and Running the Program

If using **G++ Compiler**, compile and run the program using:

```
g++ program.cpp -o program.exe # Compilation
./program                      # Execution
```

---

## 1.5 Summary

This chapter introduced **C++ programming**, covering its **history, features, and advantages**. It explained how to **set up a C++ development environment**, choose a **compiler and IDE**, and write and execute a **basic C++ program**. These fundamental concepts are **essential before diving into Object-Oriented Programming (OOP)** in later chapters.

---

## 1.6 Keywords

- **C++ Programming**
  - **Compilers**
  - **Integrated Development Environment (IDE)**
  - **Standard Template Library (STL)**
  - **Object-Oriented Programming (OOP)**
- 

## 1.7 Review Questions

1. **Who developed C++ and when?**
  2. **What are the key features of C++?**
  3. **How is C++ different from C?**
  4. **Which compiler is used for running C++ programs in Linux?**
  5. **Write a simple "Hello, World!" program in C++ and explain each part.**
  6. **Who is the creator of C++?**
    - a) Dennis Ritchie
    - b) Bjarne Stroustrup
    - c) James Gosling
    - d) Guido van Rossum
  7. **Which of the following is a C++ compiler?**
    - a) GCC
    - b) Python Interpreter
    - c) JDK
    - d) IntelliJ IDEA
- 

## 1.8 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **C++ Primer (5th Edition)** – Stanley B. Lippman
- **Accelerated C++** – Andrew Koenig and Barbara E. Moo

# Chapter 2: Fundamentals of C++ Programming

## 2.1 Objectives

By the end of this chapter, you will be able to:

- Understand **variables and data types** in C++.
- Learn about **operators and expressions**.
- Work with **control flow statements** (if-else, loops, switch).
- Use **input and output statements** efficiently.

---

## 2.2 Variables and Data Types

### 2.2.1 What is a Variable?

A **variable** is a named storage location in memory that holds a value. Variables must be **declared with a data type** before use.

### 2.2.2 Data Types in C++

C++ has several **built-in data types**:

| Data Type | Description                               | Example         |
|-----------|-------------------------------------------|-----------------|
| char      | Stores single characters                  | 'A', 'B', 'C'   |
| int       | Stores whole numbers                      | 10, -5          |
| float     | Stores decimal numbers (single precision) | 3.14, -2.5      |
| double    | Stores decimal numbers (double precision) | 3.1415926535    |
| bool      | Stores true/false values                  | true, false     |
| void      | Represents no value (used in functions)   | void function() |

Example: Declaring and Using Variables

```
#include <iostream>
using namespace std;

int main() {
    int age = 21;
    float pi = 3.14;
    char grade = 'A';
    bool isStudent = true;

    cout << "Age: " << age << endl;
    cout << "Value of Pi: " << pi << endl;
    cout << "Grade: " << grade << endl;
    cout << "Is Student? " << isStudent << endl;

    return 0;
}
```

### □ Output:

```
Age: 21
Value of Pi: 3.14
Grade: A
Is Student? 1
```

✓ `true` is displayed as `1` because `bool` values are stored as `1` (`true`) or `0` (`false`).

## 2.3 Operators in C++

C++ provides several types of **operators** for performing calculations and comparisons.

### 2.3.1 Arithmetic Operators

Used for basic math operations.

| Operator | Description         | Example            |
|----------|---------------------|--------------------|
| +        | Addition            | <code>a + b</code> |
| -        | Subtraction         | <code>a - b</code> |
| *        | Multiplication      | <code>a * b</code> |
| /        | Division            | <code>a / b</code> |
| %        | Modulus (Remainder) | <code>a % b</code> |

Example:

```
int a = 10, b = 3;
cout << "Sum: " << a + b << endl;
cout << "Remainder: " << a % b << endl;
```

### □ Output:

```
Sum: 13
Remainder: 1
```

### 2.3.2 Comparison Operators

Used to compare two values.

| Operator           | Meaning                  |
|--------------------|--------------------------|
| <code>==</code>    | Equal to                 |
| <code>!=</code>    | Not equal to             |
| <code>&gt;</code>  | Greater than             |
| <code>&lt;</code>  | Less than                |
| <code>&gt;=</code> | Greater than or equal to |
| <code>&lt;=</code> | Less than or equal to    |

Example:

```
cout << (10 > 5); // Output: 1 (true)
```

## 2.4 Control Statements in C++

Control statements help **direct the flow of execution**.

### 2.4.1 If-Else Statement

```
int age = 18;
if (age >= 18) {
    cout << "Eligible to vote";
} else {
    cout << "Not eligible";
}
```

□ **Output:** Eligible to vote

### 2.4.2 Loops in C++

Loops allow **repeating a block of code multiple times**.

#### *For Loop*

```
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}
```

□ **Output:** 1 2 3 4 5

#### *While Loop*

```
int i = 1;
while (i <= 5) {
    cout << i << " ";
    i++;
}
```

□ **Output:** 1 2 3 4 5

#### *Do-While Loop*

```
int i = 1;
do {
    cout << i << " ";
    i++;
} while (i <= 5);
```

□ **Output:** 1 2 3 4 5

---

## 2.5 Summary

- ✓ **Variables** store data, and **data types** define what kind of data can be stored.
- ✓ **Operators** perform calculations and comparisons.

- ✓ **Control statements** like **if-else** and **loops** help in decision-making and iteration.
- ✓ **Understanding these fundamentals is essential before diving into OOP concepts.**

---

## 2.6 Keywords

- **Variables**
- **Data Types**
- **Operators**
- **Control Statements**
- **Loops**

---

## 2.7 Multiple-Choice Questions (MCQs)

1. **Which of the following is NOT a data type in C++?**
  - a) `int`
  - b) `float`
  - c) `real`
  - d) `char`
2. **What is the size of an `int` in C++ (on most systems)?**
  - a) 1 byte
  - b) 2 bytes
  - c) 4 bytes
  - d) 8 bytes
3. **Which operator is used to get the remainder of a division?**
  - a) `/`
  - b) `%`
  - c) `*`
  - d) `&`
4. **Which loop executes at least once, even if the condition is false?**
  - a) For loop
  - b) While loop
  - c) Do-While loop
  - d) Infinite loop
5. **Which of the following is a comparison operator in C++?**
  - a) `=`
  - b) `==`
  - c) `+=`
  - d) `++`

### □ Keywords

- **Variables**
- **Data Types**
- **Boolean Values** (`true` / `false`)
- **Character** (`char`)

- **Floating-Point** (`float`)

---

#### □ Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **C++ Primer (5th Edition)** – Stanley B. Lippman
- **Effective C++** – Scott Meyers
- **Accelerated C++** – Andrew Koenig

## Chapter 3: Functions and Memory Management in C++

### 3.1 Objectives

By the end of this chapter, you will be able to:

- Understand **functions in C++** and their types.
- Learn about **function overloading** and **default arguments**.
- Explore **memory management in C++** using `new` and `delete`.
- Understand the concept of **pointers and dynamic memory allocation**.

---

### 3.2 Functions in C++

#### 3.2.1 What is a Function?

A **function** is a block of code that performs a specific task. It helps in:

- ✓ **Code reusability** – Write once, use multiple times.
- ✓ **Modularity** – Divides a program into manageable parts.
- ✓ **Improved readability** – Makes code structured and easy to understand.

#### 3.2.2 Function Declaration, Definition, and Calling

A function has three main components:

1. **Declaration** – Specifies the function name and return type.
2. **Definition** – Contains the actual code of the function.
3. **Function Call** – Executes the function.

*Example: Basic Function in C++*

```
#include <iostream>
using namespace std;
```



```
// Function declaration
void greet();

// Main function
int main() {
    greet(); // Function call
    return 0;
}

// Function definition
void greet() {
    cout << "Hello, welcome to C++ programming!" << endl;
}
```

#### □ **Output:**

Hello, welcome to C++ programming!

✓ The function `greet()` is declared, defined, and then called inside `main()`.

---

### 3.3 Default Arguments in Functions

A **default argument** is a value assigned to a function parameter, which is used if no value is passed during the function call.

#### *Example: Using Default Arguments*

```
#include <iostream>
using namespace std;

void display(string name = "Guest") {
    cout << "Hello, " << name << "!" << endl;
}

int main() {
    display(); // Uses default argument "Guest"
    display("Alice"); // Uses passed argument "Alice"
    return 0;
}
```

#### □ **Output:**

Hello, Guest!  
Hello, Alice!

✓ If no argument is provided, the default value "Guest" is used.

---

### 3.4 Memory Management in C++

#### 3.5.1 Dynamic Memory Allocation (`new` and `delete`)

C++ allows **manual memory management** using `new` (for allocation) and `delete` (for deallocation).

*Example: Allocating and Releasing Memory Dynamically*

```
#include <iostream>
using namespace std;

int main() {
    int* ptr = new int; // Dynamically allocate memory
    *ptr = 50;
    cout << "Value: " << *ptr << endl;

    delete ptr; // Free allocated memory
    return 0;
}
```

□ **Output:**

Value: 50

✓ `new` allocates memory, and `delete` releases it.

---

### 3.5 Summary

- ✓ **Functions help in reusability and modularity.**
- ✓ **Function overloading** allows multiple functions with the same name but different parameters.
- ✓ **Default arguments** help provide optional values in function calls.
- ✓ **Dynamic memory allocation (`new` and `delete`)** allows efficient memory usage.

---

### 3.6 Keywords

- **Function Overloading**
- **Default Arguments**
- **Dynamic Memory Allocation**
- **Pointers**
- **Memory Management**

---

### 3.7 Multiple-Choice Questions (MCQs)

1. **Which keyword is used to allocate memory dynamically in C++?**
  - a) `malloc`
  - b) `allocate`
  - c) `new`
  - d) `alloc`

2. **Which feature allows multiple functions with the same name but different parameters?**
  - a) Function overriding
  - b) Function overloading
  - c) Operator overloading
  - d) Function duplication
3. **What is the correct syntax to deallocate memory allocated by `new`?**
  - a) `free(ptr);`
  - b) `delete ptr;`
  - c) `remove ptr;`
  - d) `deallocate ptr;`
4. **What will be the output of this code?**

```
void fun(int x = 5) {
    cout << x;
}
int main() {
    fun();
}
```

  - a) 0
  - b) 5
  - c) Error
  - d) Garbage value
5. **Which of the following statements is true about default arguments?**
  - a) They must be placed at the beginning of the parameter list.
  - b) They must be placed at the end of the parameter list.
  - c) They are mandatory in all function calls.
  - d) They can be placed anywhere in the parameter list.

---

### 3.9 Further Reading

- **C++ Primer (5th Edition)** – Stanley B. Lippman
- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective C++** – Scott Meyers

## Chapter 4: Classes and Objects in C++

### 4.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what classes and objects are** in C++.
- Learn how to **define and use classes** in C++.
- Explore **data members and member functions** inside a class.
- Understand the concept of **accessing class members** using objects.

---

## 4.2 Introduction to Object-Oriented Programming (OOP)

**Object-Oriented Programming (OOP)** is a programming paradigm based on the concept of **objects**, which contain **data (attributes)** and **behavior (methods)**.

In **procedural programming**, functions operate on data. In **OOP**, data and functions are combined inside **objects**.

### 4.2.1 Key Concepts of OOP

- ✓ **Encapsulation** – Data and methods are wrapped together inside a class.
- ✓ **Abstraction** – Only essential features are exposed, hiding internal details.
- ✓ **Inheritance** – A new class can inherit properties from an existing class.
- ✓ **Polymorphism** – The same function can behave differently based on input.

C++ is an **object-oriented programming language** that allows developers to **model real-world entities using objects**.

---

## 4.3 Understanding Classes and Objects

### 4.3.1 What is a Class?

A **class** is a **blueprint** for creating objects. It defines **attributes (data members)** and **behaviors (member functions)**.

- ✓ A **class does not occupy memory** until an object is created.

### 4.3.2 What is an Object?

An **object** is an **instance of a class**. It is created based on the class structure and **occupies memory**.

---

## 4.4 Defining and Using Classes in C++

### 4.4.1 Defining a Class

A **class** in C++ is defined using the `class` keyword.

*Example: Defining a Class*

```
#include <iostream>
using namespace std;

class Car { // Class definition
public: // Access specifier
    string brand;
    int year;

    void display() { // Member function
        cout << "Car: " << brand << ", Year: " << year << endl;
    }
};
```

```
    }
};
```

✓ This defines a **Car** class with:

- **Two attributes:** `brand` (string) and `year` (integer).
- **One function:** `display()` to print car details.

#### 4.4.2 Creating Objects from a Class

Objects are created from a class **inside the `main()` function**.

*Example: Creating an Object*

```
int main() {
    Car car1; // Creating an object of the Car class

    // Assigning values to attributes
    car1.brand = "Toyota";
    car1.year = 2022;

    // Calling the member function
    car1.display();

    return 0;
}
```

□ **Output:**

```
Car: Toyota, Year: 2022
```

- ✓ The **object `car1`** is created, and **values are assigned** to its attributes.
- ✓ The **`display()` function** prints the details of `car1`.

#### 4.5 Accessing Class Members

In C++, **class members (variables and functions) are accessed using the dot `.` operator**.

**Example: Accessing Class Members**

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int age;

    void introduce() {
        cout << "My name is " << name << " and I am " << age << " years
old." << endl;
    }
};
```

```
int main() {
    Student s1; // Object creation
    s1.name = "Alice";
    s1.age = 20;

    s1.introduce(); // Calling member function

    return 0;
}
```

#### □ Output:

My name is Alice and I am 20 years old.

✓ **s1.name = "Alice";** assigns a value to the attribute.

✓ **s1.introduce();** calls the member function to print details.

## 4.6 Memory Allocation of Objects in C++

When an **object** is created from a class, memory is allocated **only for its attributes**.  
**Member functions are stored separately in memory.**

### Example: Multiple Objects and Independent Memory

```
#include <iostream>
using namespace std;

class Employee {
public:
    string name;
    int salary;
};

int main() {
    Employee e1, e2; // Two objects created

    e1.name = "John";
    e1.salary = 50000;

    e2.name = "Emma";
    e2.salary = 60000;

    cout << "Employee 1: " << e1.name << ", Salary: " << e1.salary << endl;
    cout << "Employee 2: " << e2.name << ", Salary: " << e2.salary << endl;

    return 0;
}
```

#### □ Output:

Employee 1: John, Salary: 50000  
 Employee 2: Emma, Salary: 60000

- ✓ Each object has its own separate memory allocation.
- ✓ Changing e1 does not affect e2, ensuring data independence.

## 4.7 Summary

- ✓ A class is a **blueprint for objects**, containing **attributes (variables)** and **methods (functions)**.
- ✓ An **object** is an **instance of a class** that **occupies memory**.
- ✓ **Class members are accessed using the . operator.**
- ✓ **Objects have independent memory** for attributes but share member functions.

## 4.8 Keywords

- **Class**
- **Object**
- **Member Functions**
- **Attributes (Data Members)**
- **Dot Operator (.)**

## 4.9 Multiple-Choice Questions (MCQs)

1. **Which keyword is used to define a class in C++?**
  - a) struct
  - b) class
  - c) object
  - d) define
2. **Which of the following is true about objects in C++?**
  - a) Objects are templates for creating classes.
  - b) Objects are instances of a class.
  - c) Objects do not occupy memory.
  - d) Objects cannot call functions.
3. **How do you access a member function of a class?**
  - a) object->function()
  - b) object.function()
  - c) class.function()
  - d) function.object()
4. **What is the main difference between a class and an object?**
  - a) A class is an instance of an object.
  - b) A class defines properties, while an object stores values.
  - c) A class occupies memory, but an object does not.
  - d) There is no difference.
5. **What happens when multiple objects are created from the same class?**
  - a) They share memory for data members.
  - b) Each object gets its own memory for attributes.
  - c) Only one object can be created.
  - d) The class gets deleted automatically.

---

#### 4.10 Further Reading

- **C++ Primer (5th Edition)** – Stanley B. Lippman
- **The C++ Programming Language** – Bjarne Stroustrup
- **Object-Oriented Programming with C++** – E. Balagurusamy

## Chapter 5: Constructors and Destructors in C++

### 5.1 Objectives

By the end of this chapter, you will be able to:

- Understand **constructors** and their purpose in C++.
- Learn about **different types of constructors** (default, parameterized, copy).
- Understand **destructors** and their role in memory management.
- Learn how **constructors and destructors work automatically** when objects are created and destroyed.

---

### 5.2 Introduction to Constructors and Destructors

#### 5.2.1 What is a Constructor?

A **constructor** is a **special function** that **initializes an object** automatically when it is created.

- ✓ Same name as the class.
- ✓ No return type (not even `void`).
- ✓ Called automatically when an object is created.

*Example: Basic Constructor*

```
#include <iostream>
using namespace std;
```



```

class Car {
public:
    string brand;

    // Constructor
    Car() {
        cout << "Constructor called!" << endl;
        brand = "Toyota";
    }

    void display() {
        cout << "Car brand: " << brand << endl;
    }
};

int main() {
    Car c1; // Object created, constructor called automatically
    c1.display();
    return 0;
}

```

#### □ Output:

```

Constructor called!
Car brand: Toyota

```

✓ The **constructor initializes the brand variable** automatically when `c1` is created.

## 5.3 Types of Constructors

### 5.3.1 Default Constructor

A **default constructor** takes **no arguments** and initializes objects with **default values**.

#### *Example: Default Constructor*

```

class Student {
public:
    Student() {
        cout << "Default Constructor Called" << endl;
    }
};

int main() {
    Student s1; // Constructor runs automatically
    return 0;
}

```

#### □ Output:

```

Default Constructor Called

```

✓ The constructor runs **without any arguments** when `s1` is created.

### 5.3.2 Parameterized Constructor

A **parameterized constructor** allows passing **values** while creating objects.

*Example: Parameterized Constructor*

```
class Employee {
public:
    string name;
    int salary;

    // Parameterized Constructor
    Employee(string empName, int empSalary) {
        name = empName;
        salary = empSalary;
    }

    void display() {
        cout << "Employee: " << name << ", Salary: $" << salary << endl;
    }
};

int main() {
    Employee e1("Alice", 50000);
    Employee e2("Bob", 60000);

    e1.display();
    e2.display();
    return 0;
}
```

□ **Output:**

```
Employee: Alice, Salary: $50000
Employee: Bob, Salary: $60000
```

✓ The **constructor assigns values** when objects e1 and e2 are created.

### 5.3.3 Copy Constructor

A **copy constructor** creates a **new object as an exact copy of an existing object**.

*Example: Copy Constructor*

```
class Number {
public:
    int value;

    // Parameterized Constructor
    Number(int v) {
        value = v;
    }

    // Copy Constructor
    Number(const Number &obj) {
        value = obj.value;
    }
}
```

```

    }

    void display() {
        cout << "Value: " << value << endl;
    }
};

int main() {
    Number n1(10);
    Number n2 = n1; // Copy constructor is called

    n1.display();
    n2.display();
    return 0;
}

```

#### □ Output:

```

Value: 10
Value: 10

```

✓  $n_2$  is a copy of  $n_1$  because of the **copy constructor**.

## 5.4 What is a Destructor?

A **destructor** is a special function that runs **when an object is destroyed**.

- ✓ Same name as the class, preceded by ~ (tilde).
- ✓ No parameters and no return type.
- ✓ Called automatically when an object goes out of scope.

### *Example: Destructor in Action*

```

class Test {
public:
    Test() {
        cout << "Constructor called!" << endl;
    }

    ~Test() {
        cout << "Destructor called!" << endl;
    }
};

int main() {
    Test t1; // Constructor runs automatically
    return 0;
} // Destructor runs when main() ends

```

#### □ Output:

```

Constructor called!
Destructor called!

```

✓ The **destructor runs automatically** when  $t_1$  goes out of scope.

---

## 5.5 Constructor vs Destructor: Key Differences

| Feature            | Constructor                                 | Destructor                                         |
|--------------------|---------------------------------------------|----------------------------------------------------|
| <b>Purpose</b>     | Initializes an object                       | Cleans up before object is destroyed               |
| <b>Name</b>        | Same as class name                          | Same as class name but prefixed with ~             |
| <b>Parameters</b>  | Can have parameters                         | No parameters                                      |
| <b>Return Type</b> | No return type                              | No return type                                     |
| <b>Calls</b>       | Called automatically when object is created | Called automatically when object goes out of scope |

---

## 5.6 Summary

- ✓ **Constructors** initialize objects **automatically** when created.
  - ✓ **Types of constructors:** Default, Parameterized, and Copy Constructors.
  - ✓ **Destructors** release resources when objects go out of scope.
  - ✓ **A destructor is called automatically at the end of an object's lifetime.**
- 

## 5.7 Keywords

- **Constructor**
  - **Destructor**
  - **Copy Constructor**
  - **Default Constructor**
  - **Parameterized Constructor**
- 

## 5.8 Multiple-Choice Questions (MCQs)

1. **Which of the following is true about constructors in C++?**
  - a) A constructor must have a return type.
  - b) A constructor is called automatically when an object is created.
  - c) A constructor is called manually.
  - d) A constructor must be `static`.
2. **What is the purpose of a destructor?**
  - a) To allocate memory dynamically.
  - b) To initialize object values.
  - c) To clean up when an object is destroyed.
  - d) To copy one object into another.
3. **Which of the following symbols represents a destructor?**
  - a) \*
  - b) &
  - c) ~
  - d) ::
4. **What does a copy constructor do?**
  - a) Initializes a new object with default values.
  - b) Destroys an object.

- c) Copies data from an existing object.
  - d) Allocates memory dynamically.
5. **Which statement is correct about parameterized constructors?**
- a) They must always be defined inside the `main()` function.
  - b) They allow passing values while creating an object.
  - c) They cannot take any parameters.
  - d) They do not initialize variables.

---

## 5.9 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Object-Oriented Programming with C++** – E. Balagurusamy
- **C++ Primer (5th Edition)** – Stanley B. Lippman

# Chapter 6: Inheritance in C++

## 6.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what inheritance is** and its advantages.
- Learn about **different types of inheritance** in C++.
- Understand how **base and derived classes work**.
- Explore the concept of **function overriding** in inheritance.

---

## 6.2 What is Inheritance?

**Inheritance** is a fundamental concept in **Object-Oriented Programming (OOP)** that allows a new class (derived class) to **reuse and extend** the properties and behavior of an existing class (base class).

### 6.2.1 Why Use Inheritance?

- ✓ **Code Reusability** – Avoid rewriting the same code in multiple classes.
- ✓ **Modularity** – Organize related classes in a hierarchical structure.
- ✓ **Extensibility** – Easily add new features to existing code.

---

## 6.3 Types of Inheritance in C++

C++ supports **following types of inheritance**:

| Inheritance Type                | Description                                                |
|---------------------------------|------------------------------------------------------------|
| <b>Single Inheritance</b>       | One class inherits from another class.                     |
| <b>Multiple Inheritance</b>     | A class inherits from more than one base class.            |
| <b>Multilevel Inheritance</b>   | A class is derived from another derived class.             |
| <b>Hierarchical Inheritance</b> | Multiple derived classes inherit from the same base class. |

---

## 6.4 Single Inheritance

### 6.4.1 Understanding Base and Derived Classes

- A **base class** is the class from which properties and methods are inherited.
- A **derived class** is the class that inherits from the base class.

*Example: Single Inheritance*

```
#include <iostream>
using namespace std;

// Base class
class Animal {
public:
    void sound() {
        cout << "Animals make sounds" << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void bark() {
        cout << "Dog barks" << endl;
    }
};

int main() {
    Dog d1;
    d1.sound(); // Inherited from Animal
    d1.bark();  // Defined in Dog
    return 0;
}
```

□ **Output:**

Animals make sounds

Dog barks

✓ **Dog** inherits the `sound()` function from **Animal**.

✓ `bark()` is specific to the **Dog** class.

## 6.5 Multiple Inheritance

A derived class inherits from **two or more base classes**.

*Example: Multiple Inheritance*

```
#include <iostream>
using namespace std;

// Base class 1
class Parent1 {
public:
    void show1() {
        cout << "Parent1 function" << endl;
    }
};

// Base class 2
class Parent2 {
public:
    void show2() {
        cout << "Parent2 function" << endl;
    }
};

// Derived class
class Child : public Parent1, public Parent2 {
};

int main() {
    Child obj;
    obj.show1();
    obj.show2();
    return 0;
}
```

□ **Output:**

```
Parent1 function
Parent2 function
```

✓ **Child class** inherits methods from **both Parent1 and Parent2**.

## 6.6 Multilevel Inheritance

A derived class inherits from another derived class.

*Example: Multilevel Inheritance*

```

#include <iostream>
using namespace std;

// Base class
class Grandparent {
public:
    void showGrandparent() {
        cout << "Grandparent class" << endl;
    }
};

// Derived class 1
class Parent : public Grandparent {
public:
    void showParent() {
        cout << "Parent class" << endl;
    }
};

// Derived class 2
class Child : public Parent {
public:
    void showChild() {
        cout << "Child class" << endl;
    }
};

int main() {
    Child c;
    c.showGrandparent();
    c.showParent();
    c.showChild();
    return 0;
}

```

□ **Output:**

```

Grandparent class
Parent class
Child class

```

✓ **The Child class inherits from Parent, and Parent inherits from Grandparent.**

## 6.7 Function Overriding in Inheritance

Function overriding allows a **derived class to redefine a function from the base class.**

*Example: Function Overriding*

```

#include <iostream>
using namespace std;

// Base class
class Animal {
public:

```



```

        void sound() {
            cout << "Animals make different sounds" << endl;
        }
    };

    // Derived class
    class Dog : public Animal {
    public:
        void sound() { // Overriding function
            cout << "Dog barks" << endl;
        }
    };

    int main() {
        Dog d;
        d.sound(); // Calls overridden function
        return 0;
    }

```

#### □ Output:

Dog barks

✓ `sound()` is **overridden** in the **Dog class**, so it replaces the base class version.

## 6.8 Summary

✓ **Inheritance** allows a new class (derived class) to reuse and extend the properties of an existing class (base class).

✓ **Single inheritance** enables a class to inherit from one base class.

✓ **Multiple inheritance** allows a class to inherit from more than one base class.

✓ **Multilevel inheritance** creates a chain of inheritance.

✓ **Function overriding** allows a derived class to modify a function from the base class.

## 6.9 Keywords

- **Inheritance**
- **Base Class**
- **Derived Class**
- **Function Overriding**
- **Multiple Inheritance**

## 6.10 Multiple-Choice Questions (MCQs)

1. **Which type of inheritance allows a class to inherit from two or more base classes?**
  - a) Single Inheritance
  - b) Multiple Inheritance
  - c) Multilevel Inheritance
  - d) Hierarchical Inheritance

2. **Which keyword is used to define inheritance in C++?**
    - a) inherits
    - b) extends
    - c) public
    - d) class
  3. **Which function can be redefined in a derived class?**
    - a) Constructor
    - b) Destructor
    - c) Virtual Function
    - d) All of the above
  4. **Which of the following is NOT a type of inheritance in C++?**
    - a) Single
    - b) Multilevel
    - c) Bidirectional
    - d) Hybrid
  5. **What happens if a derived class does not override a function from the base class?**
    - a) It results in an error.
    - b) The base class function is inherited and used.
    - c) The derived class cannot be created.
    - d) None of the above.
- 

### 6.11 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Object-Oriented Programming with C++** – E. Balagurusamy
- **C++ Primer (5th Edition)** – Stanley B. Lippman

## Chapter 7: Multiple and Multilevel Inheritance in C++

### 7.1 Objectives

By the end of this chapter, you will be able to:

- Understand **Multiple Inheritance** and how a class can inherit from more than one base class.
  - Learn about **Multilevel Inheritance**, where a derived class inherits from another derived class.
  - Explore the **advantages and challenges** of using multiple and multilevel inheritance.
  - Understand the **Diamond Problem** and how to solve it using **virtual inheritance**.
-

## 7.2 Multiple Inheritance in C++

### 7.2.1 What is Multiple Inheritance?

**Multiple Inheritance** is a type of inheritance where a **single derived class** inherits from **two or more base classes**.

- ✓ Allows reusing features of multiple classes in one class.
- ✓ Increases code modularity and reusability.
- ✓ May lead to ambiguity issues (Diamond Problem).

### 7.2.2 Example: Multiple Inheritance

```
#include <iostream>
using namespace std;

// Base class 1
class Parent1 {
public:
    void show1() {
        cout << "Parent1 function" << endl;
    }
};

// Base class 2
class Parent2 {
public:
    void show2() {
        cout << "Parent2 function" << endl;
    }
};

// Derived class inheriting from both Parent1 and Parent2
class Child : public Parent1, public Parent2 {
public:
    void showChild() {
        cout << "Child function" << endl;
    }
};

int main() {
    Child obj;
    obj.show1();
    obj.show2();
    obj.showChild();
    return 0;
}
```

#### □ Output:

```
Parent1 function
Parent2 function
Child function
```

- ✓ Child class inherits functions from both Parent1 and Parent2.

---

## 7.3 The Diamond Problem in Multiple Inheritance

### 7.3.1 What is the Diamond Problem?

When **two base classes** inherit from the same **grandparent class**, and the **derived class inherits from both**, **ambiguity arises**.

*Example: The Diamond Problem*

```
#include <iostream>
using namespace std;

class Grandparent {
public:
    void show() {
        cout << "Grandparent function" << endl;
    }
};

class Parent1 : public Grandparent {};

class Parent2 : public Grandparent {};

// Child inherits from both Parent1 and Parent2
class Child : public Parent1, public Parent2 {};

int main() {
    Child obj;
    obj.show(); // ERROR: Ambiguity!
    return 0;
}
```

❑ **Error:** request for member 'show' is ambiguous

✓ **The compiler doesn't know whether to call show() from Parent1 or Parent2.**

---

### 7.3.2 Solving the Diamond Problem Using Virtual Inheritance

✓ **Solution:** Use **virtual inheritance** to ensure only one copy of Grandparent exists.

*Example: Virtual Inheritance*

```
#include <iostream>
using namespace std;

class Grandparent {
public:
    void show() {
        cout << "Grandparent function" << endl;
    }
};

// Virtual inheritance to prevent ambiguity
class Parent1 : virtual public Grandparent {};
class Parent2 : virtual public Grandparent {};
```

```

class Child : public Parent1, public Parent2 {};

int main() {
    Child obj;
    obj.show(); // No ambiguity!
    return 0;
}

```

#### □ Output:

Grandparent function

✓ Using `virtual` ensures only one copy of `Grandparent` is inherited.

## 7.4 Multilevel Inheritance in C++

### 7.4.1 What is Multilevel Inheritance?

**Multilevel Inheritance** is a type of inheritance where a **derived class inherits from another derived class**.

✓ Creates a chain of inheritance.

✓ Each derived class gets features from all preceding classes.

### 7.4.2 Example: Multilevel Inheritance

```

#include <iostream>
using namespace std;

// Base class
class Grandparent {
public:
    void showGrandparent() {
        cout << "Grandparent function" << endl;
    }
};

// Derived class 1
class Parent : public Grandparent {
public:
    void showParent() {
        cout << "Parent function" << endl;
    }
};

// Derived class 2
class Child : public Parent {
public:
    void showChild() {
        cout << "Child function" << endl;
    }
};

int main() {

```

```

    Child obj;
    obj.showGrandparent();
    obj.showParent();
    obj.showChild();
    return 0;
}

```

#### □ Output:

```

Grandparent function
Parent function
Child function

```

- ✓ **The Child class inherits from Parent, and Parent inherits from Grandparent.**
- ✓ **This creates a chain of inheritance.**

---

## 7.5 Advantages and Disadvantages of Multiple and Multilevel Inheritance

| Advantages                              | Disadvantages                                    |
|-----------------------------------------|--------------------------------------------------|
| Code Reusability                        | Complexity in managing dependencies              |
| Reduces Code Duplication                | Increases chances of ambiguity (Diamond Problem) |
| Organizes code in a hierarchical manner | Harder to debug in large projects                |

---

## 7.6 Summary

- ✓ **Multiple Inheritance** allows a derived class to inherit from multiple base classes.
- ✓ **Multilevel Inheritance** allows a derived class to inherit from another derived class.
- ✓ **The Diamond Problem occurs in Multiple Inheritance**, causing ambiguity.
- ✓ **Virtual Inheritance solves the Diamond Problem** by ensuring only one copy of the base class exists.

---

## 7.7 Keywords

- **Multiple Inheritance**
- **Multilevel Inheritance**
- **Virtual Inheritance**
- **Diamond Problem**
- **Ambiguity**

---

## 7.8 Multiple-Choice Questions (MCQs)

1. **Which type of inheritance allows a class to inherit from more than one base class?**
  - a) Single Inheritance
  - b) Multilevel Inheritance

- c) Multiple Inheritance
  - d) Hierarchical Inheritance
  - 2. **What issue can arise due to Multiple Inheritance?**
    - a) Memory leak
    - b) Diamond Problem
    - c) Syntax Error
    - d) Stack Overflow
  - 3. **Which keyword is used to prevent ambiguity in Multiple Inheritance?**
    - a) `static`
    - b) `friend`
    - c) `virtual`
    - d) `private`
  - 4. **Which inheritance type forms a chain where a class inherits from a derived class?**
    - a) Multiple Inheritance
    - b) Hybrid Inheritance
    - c) Multilevel Inheritance
    - d) Single Inheritance
  - 5. **What is the solution to the Diamond Problem in C++?**
    - a) Use `static` functions
    - b) Use `friend` functions
    - c) Use `virtual` inheritance
    - d) Use `private` inheritance
- 

## 7.9 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Object-Oriented Programming with C++** – E. Balagurusamy
- **C++ Primer (5th Edition)** – Stanley B. Lippman

# Chapter 8: Polymorphism in C++

## 8.1 Objectives

By the end of this chapter, you will be able to:

- Understand the **concept of polymorphism** in C++.

- Learn about **compile-time (static) polymorphism** and **run-time (dynamic) polymorphism**.
- Explore **function overloading** and **operator overloading** for static polymorphism.
- Understand **function overriding** and the role of **virtual functions** for dynamic polymorphism.

---

## 8.2 What is Polymorphism?

**Polymorphism** means "many forms" and allows a function, operator, or object to **behave differently in different situations**.

- ✓Increases **code flexibility** and **reusability**.
- ✓Allows **same function name** but **different implementations**.
- ✓Helps in **object-oriented programming** by supporting **overloading and overriding**.

---

## 8.3 Types of Polymorphism in C++

| Type of Polymorphism                      | Description                                      | Examples                                   |
|-------------------------------------------|--------------------------------------------------|--------------------------------------------|
| <b>Compile-time (Static) Polymorphism</b> | Function behavior is determined at compile-time. | Function Overloading, Operator Overloading |
| <b>Run-time (Dynamic) Polymorphism</b>    | Function behavior is determined at run-time.     | Function Overriding, Virtual Functions     |

---

## 8.4 Compile-Time (Static) Polymorphism

### 8.4.1 Function Overloading

- ✓**Same function name**, but **different parameters**.
- ✓The function **executes based on argument types** at compile-time.

*Example: Function Overloading*

```
#include <iostream>
using namespace std;

class MathOperations {
public:
    // Function with two int parameters
    int add(int a, int b) {
        return a + b;
    }

    // Function with two double parameters
    double add(double a, double b) {
        return a + b;
    }

    // Function with three parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
};
```



```
int main() {
    MathOperations obj;
    cout << "Sum (int): " << obj.add(5, 10) << endl;
    cout << "Sum (double): " << obj.add(2.5, 3.7) << endl;
    cout << "Sum (three int): " << obj.add(1, 2, 3) << endl;
    return 0;
}
```

#### □ Output:

```
Sum (int): 15
Sum (double): 6.2
Sum (three int): 6
```

✓ **Overloaded functions** work based on **different argument types**.

## 8.4.2 Operator Overloading

✓ **Allows defining custom behavior for operators (+, -, \*, etc.).**

✓ **Makes objects work like built-in data types.**

### *Example: Operator Overloading (+)*

```
#include <iostream>
using namespace std;

class Complex {
public:
    int real, imag;

    Complex(int r, int i) {
        real = r;
        imag = i;
    }

    // Overloading the + operator
    Complex operator+(Complex const &obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3, 2), c2(1, 7);
    Complex c3 = c1 + c2; // Uses operator overloading
    c3.display();
    return 0;
}
```

#### □ Output:

```
4 + 9i
```

✓The + operator now works with complex objects using operator overloading.

## 8.5 Run-Time (Dynamic) Polymorphism

### 8.5.1 Function Overriding

✓A function in a base class is redefined in a derived class.

✓Method resolution happens at run-time.

*Example: Function Overriding*

```
#include <iostream>
using namespace std;

class Animal {
public:
    void sound() {
        cout << "Animals make sounds" << endl;
    }
};

// Derived class
class Dog : public Animal {
public:
    void sound() { // Overriding function
        cout << "Dog barks" << endl;
    }
};

int main() {
    Dog d;
    d.sound(); // Calls overridden function
    return 0;
}
```

□ **Output:**

Dog barks

✓The sound() function is overridden in the Dog class.

### 8.5.2 Virtual Functions

✓A virtual function in a base class allows the derived class to **override it dynamically**.

✓Ensures function calls are resolved at run-time (late binding).

*Example: Virtual Functions*

```
#include <iostream>
using namespace std;

class Animal {
public:
    virtual void sound() { // Virtual function
```

```

        cout << "Animal makes a sound" << endl;
    }
};

class Dog : public Animal {
public:
    void sound() override { // Overridden function
        cout << "Dog barks" << endl;
    }
};

int main() {
    Animal* a;
    Dog d;
    a = &d;
    a->sound(); // Calls overridden function in Dog
    return 0;
}

```

#### □ Output:

Dog barks

✓ Using `virtual` allows dynamic function selection at run-time.

## 8.6 Summary

- ✓ **Polymorphism** allows functions, operators, and objects to take multiple forms.
- ✓ **Compile-time polymorphism** (function overloading, operator overloading) is resolved at compile-time.
- ✓ **Run-time polymorphism** (function overriding, virtual functions) is resolved at run-time.
- ✓ **Virtual functions enable dynamic dispatching of function calls in inheritance.**

## 8.7 Keywords

- **Polymorphism**
- **Function Overloading**
- **Operator Overloading**
- **Function Overriding**
- **Virtual Functions**

## 8.8 Multiple-Choice Questions (MCQs)

1. **Which type of polymorphism allows a function to have multiple implementations based on the number/type of arguments?**
  - a) Function Overriding
  - b) Function Overloading
  - c) Operator Overloading
  - d) Data Abstraction

2. **Which of the following enables run-time polymorphism in C++?**
  - a) Function Overloading
  - b) Operator Overloading
  - c) Virtual Functions
  - d) Constructor Overloading
3. **What is the output of the following code?**

```
class A {
public:
    virtual void show() { cout << "Base class"; }
};
class B : public A {
public:
    void show() { cout << "Derived class"; }
};
int main() {
    A* obj;
    B b;
    obj = &b;
    obj->show();
    return 0;
}
```

- a) Base class
  - b) Derived class
  - c) Compilation Error
  - d) Runtime Error
4. **Which operator cannot be overloaded in C++?**
  - a) +
  - b) =
  - c) sizeof
  - d) <<
5. **Why do we use virtual functions?**
  - a) To achieve function overloading
  - b) To enable function overriding at runtime
  - c) To overload operators
  - d) To improve compilation speed

---

## 8.9 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Object-Oriented Programming with C++** – E. Balagurusamy
- **C++ Primer (5th Edition)** – Stanley B. Lippman

# Chapter 9: Applications of Polymorphism in C++

## 9.1 Objectives

By the end of this chapter, you will be able to:

- Understand **how polymorphism is applied** in real-world programming.
- Learn **practical applications of function overloading and function overriding**.
- Explore **operator overloading in mathematical computations**.
- Understand how **virtual functions** are used in modern software development.

---

## 9.2 Introduction to Applications of Polymorphism

**Polymorphism** allows objects, functions, and operators to take multiple forms, improving **code reusability and flexibility**. It plays a crucial role in:

- ✓ **GUI development** – Polymorphic functions help in rendering UI components.
- ✓ **Gaming engines** – Different objects (characters, enemies) share the same interface.
- ✓ **Mathematical computations** – Operator overloading simplifies calculations.
- ✓ **Database management systems (DBMS)** – Virtual functions enable dynamic queries.

---

## 9.3 Application of Function Overloading

### 9.3.1 Function Overloading in Mathematical Computation

✓ Function overloading simplifies the **implementation of mathematical operations** for different data types.

*Example: Function Overloading for a Calculator*

```
#include <iostream>
using namespace std;

class Calculator {
public:
    // Overloaded add function for integers
    int add(int a, int b) {
        return a + b;
    }

    // Overloaded add function for double values
    double add(double a, double b) {
        return a + b;
    }
};

int main() {
    Calculator calc;
    cout << "Sum (int): " << calc.add(5, 10) << endl;
    cout << "Sum (double): " << calc.add(3.5, 2.7) << endl;
    return 0;
}
```

```
}
```

#### □ Output:

```
Sum (int): 15
Sum (double): 6.2
```

✓ **Function overloading allows `add()` to work for both integers and doubles.**

## 9.4 Application of Operator Overloading

✓ **Operator overloading** enables objects to use arithmetic operators (+, -, \*, etc.).

✓ **It is widely used in scientific computing, graphics, and game development.**

*Example: Operator Overloading for Complex Numbers*

```
#include <iostream>
using namespace std;

class Complex {
public:
    int real, imag;

    Complex(int r, int i) {
        real = r;
        imag = i;
    }

    // Overloading the + operator
    Complex operator+(Complex const &obj) {
        return Complex(real + obj.real, imag + obj.imag);
    }

    void display() {
        cout << real << " + " << imag << "i" << endl;
    }
};

int main() {
    Complex c1(3, 2), c2(1, 7);
    Complex c3 = c1 + c2; // Uses operator overloading
    c3.display();
    return 0;
}
```

#### □ Output:

```
4 + 9i
```

✓ **Operator overloading allows `+` to work for `Complex` objects, just like integers.**

## 9.5 Application of Function Overriding

✓ **Function overriding** is used in real-world applications such as **game development, GUI programming, and API design.**

*Example: Function Overriding in a Game*

```
#include <iostream>
using namespace std;

class Character {
public:
    virtual void attack() { // Virtual function
        cout << "Character attacks!" << endl;
    }
};

class Warrior : public Character {
public:
    void attack() override {
        cout << "Warrior swings sword!" << endl;
    }
};

class Archer : public Character {
public:
    void attack() override {
        cout << "Archer shoots an arrow!" << endl;
    }
};

int main() {
    Character* c1;
    Warrior w;
    Archer a;

    c1 = &w;
    c1->attack(); // Calls Warrior's attack()

    c1 = &a;
    c1->attack(); // Calls Archer's attack()

    return 0;
}
```

□ **Output:**

```
Warrior swings sword!
Archer shoots an arrow!
```

✓ **Dynamic dispatch via virtual functions allows different attack behaviors.**

---

## 9.6 Application of Virtual Functions in Real-World Software

✓ **Virtual functions** enable polymorphism in **database management, artificial intelligence, and simulations.**

### Example: Virtual Functions in a Database System

```
#include <iostream>
using namespace std;

class Database {
public:
    virtual void query() {
        cout << "Executing general query..." << endl;
    }
};

class MySQL : public Database {
public:
    void query() override {
        cout << "Executing MySQL-specific query..." << endl;
    }
};

class PostgreSQL : public Database {
public:
    void query() override {
        cout << "Executing PostgreSQL-specific query..." << endl;
    }
};

int main() {
    Database* db;
    MySQL my;
    PostgreSQL pg;

    db = &my;
    db->query(); // Calls MySQL's query()

    db = &pg;
    db->query(); // Calls PostgreSQL's query()

    return 0;
}
```

#### □ Output:

```
Executing MySQL-specific query...
Executing PostgreSQL-specific query...
```

✓ **Virtual functions enable dynamic query execution for different databases.**

---

## 9.7 Summary

- ✓ **Function overloading** is used in **mathematical computations and calculators**.
  - ✓ **Operator overloading** is used in **scientific computing and game development**.
  - ✓ **Function overriding** is widely used in **game engines and UI frameworks**.
  - ✓ **Virtual functions** allow **dynamic behavior in software like databases and AI models**.
-



## 9.8 Keywords

- **Function Overloading**
- **Operator Overloading**
- **Function Overriding**
- **Virtual Functions**
- **Polymorphism in Software Development**

---

## 9.9 Multiple-Choice Questions (MCQs)

1. **Which of the following is an application of function overloading?**
  - a) Defining multiple functions with the same name but different parameters
  - b) Overriding a function in a derived class
  - c) Using pointers to access functions dynamically
  - d) None of the above
2. **How does operator overloading benefit scientific applications?**
  - a) It reduces memory usage
  - b) It allows built-in operators to work with user-defined types
  - c) It avoids using loops
  - d) It makes functions static
3. **What is the primary advantage of virtual functions in database systems?**
  - a) Faster execution time
  - b) Reducing redundant function calls
  - c) Enabling different implementations for different database types
  - d) Storing data in compressed format
4. **Which of the following is an example of function overriding?**
  - a) Defining multiple functions with the same name in the same class
  - b) Redefining a function in a derived class with the same signature
  - c) Using the + operator for object addition
  - d) Implementing different query languages
5. **Which programming concept allows different classes to use the same method name but with different implementations?**
  - a) Encapsulation
  - b) Abstraction
  - c) Polymorphism
  - d) Inheritance

---

## 9.10 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective C++** – Scott Meyers
- **Object-Oriented Programming with C++** – E. Balagurusamy

# Chapter 10: Abstract Classes and Interfaces in C++

## 10.1 Objectives

By the end of this chapter, you will be able to:

- Understand **abstract classes** and their importance in C++.
- Learn about **pure virtual functions** and how they enforce method implementation in derived classes.
- Differentiate between **abstract classes and interfaces**.
- Explore real-world **applications of abstract classes** in software design.

---

## 10.2 Introduction to Abstract Classes

An **abstract class** is a class that **cannot be instantiated directly**. It serves as a **blueprint** for other classes and ensures that **certain functions are implemented in derived classes**.

- ✓ **Used to enforce a common interface across multiple derived classes.**
- ✓ **Contains at least one pure virtual function.**
- ✓ **Used in frameworks, API design, and real-world software applications.**

### 10.2.1 Why Use Abstract Classes?

- ✓ **Encapsulation of common behavior** across different types of objects.
- ✓ **Forces derived classes to implement essential methods.**
- ✓ **Promotes modular and scalable code design.**

---

## 10.3 Pure Virtual Functions

A **pure virtual function** is a function that must be **overridden** in a derived class.

- ✓ **Declared using `= 0` syntax.**
- ✓ **Makes a class abstract (non-instantiable).**
- ✓ **Ensures derived classes provide their own implementation.**

*Example: Defining an Abstract Class*

```
#include <iostream>
using namespace std;

// Abstract class
class Shape {
public:
    virtual void draw() = 0; // Pure virtual function
```

```

};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing Circle" << endl;
    }
};

class Rectangle : public Shape {
public:
    void draw() override {
        cout << "Drawing Rectangle" << endl;
    }
};

int main() {
    Shape* s1 = new Circle();
    Shape* s2 = new Rectangle();

    s1->draw(); // Calls Circle's draw()
    s2->draw(); // Calls Rectangle's draw()

    delete s1;
    delete s2;
    return 0;
}

```

#### □ Output:

```

Drawing Circle
Drawing Rectangle

```

- ✓ The draw() function is **overridden in each derived class**.
- ✓ The Shape class **cannot be instantiated directly**.

---

### 10.4 Abstract Classes vs. Interfaces in C++

| Feature                        | Abstract Class                                                    | Interface                                                       |
|--------------------------------|-------------------------------------------------------------------|-----------------------------------------------------------------|
| <b>Definition</b>              | A class with at least one pure virtual function.                  | A fully abstract class with only pure virtual functions.        |
| <b>Member Variables</b>        | Can have data members.                                            | Cannot have data members.                                       |
| <b>Function Implementation</b> | Can have both implemented and pure virtual functions.             | Only pure virtual functions (no function implementation).       |
| <b>Use Case</b>                | Used when some behavior needs to be shared among derived classes. | Used when multiple unrelated classes need to follow a contract. |

---

### 10.5 Real-World Applications of Abstract Classes

- ✓ **Graphical User Interfaces (GUI)** – Enforcing a common structure for UI elements.
- ✓ **Gaming Engines** – Defining base classes for different characters and game objects.
- ✓ **Database Systems** – Ensuring different database implementations provide standard methods.

---

## 10.6 Example: Abstract Class in a Banking System

### *Scenario*

A **banking system** needs multiple account types (`SavingsAccount`, `CurrentAccount`). Each account must have a `calculateInterest()` method, but the formula differs.

### *Implementation*

```
#include <iostream>
using namespace std;

// Abstract class
class BankAccount {
public:
    virtual void calculateInterest() = 0;    // Pure virtual function
};

class SavingsAccount : public BankAccount {
public:
    void calculateInterest() override {
        cout << "Calculating interest for Savings Account" << endl;
    }
};

class CurrentAccount : public BankAccount {
public:
    void calculateInterest() override {
        cout << "No interest for Current Account" << endl;
    }
};

int main() {
    BankAccount* acc1 = new SavingsAccount();
    BankAccount* acc2 = new CurrentAccount();

    acc1->calculateInterest();
    acc2->calculateInterest();

    delete acc1;
    delete acc2;
    return 0;
}
```

### □ **Output:**

```
Calculating interest for Savings Account
No interest for Current Account
```

- ✓ **Each account type provides its own implementation of `calculateInterest()`.**
- ✓ **`BankAccount` ensures all accounts follow a common interface.**

---

## 10.7 Summary

- ✓ **Abstract classes** cannot be instantiated and serve as blueprints for derived classes.
- ✓ **A pure virtual function** must be overridden in a derived class.

- ✓ Abstract classes help in enforcing a common interface across multiple related classes.
- ✓ Interfaces are fully abstract classes used for defining contracts across unrelated classes.

## 10.8 Keywords

- Abstract Class
- Pure Virtual Function
- Virtual Functions
- Inheritance
- Interface

## 10.9 Multiple-Choice Questions (MCQs)

1. **Which statement about abstract classes is correct?**
  - a) An abstract class cannot have any implemented methods.
  - b) Abstract classes must have only one derived class.
  - c) Abstract classes must have at least one pure virtual function.
  - d) Abstract classes cannot be inherited.
2. **What is the purpose of a pure virtual function?**
  - a) To allow function overloading.
  - b) To force derived classes to implement the function.
  - c) To allow constructors to be virtual.
  - d) To define a class without data members.
3. **Which of the following is a difference between an abstract class and an interface?**
  - a) Abstract classes cannot have pure virtual functions.
  - b) Interfaces cannot have implemented methods.
  - c) Interfaces can store data members.
  - d) Abstract classes cannot have constructors.
4. **What happens if a derived class does not override a pure virtual function?**
  - a) The program compiles successfully.
  - b) The derived class becomes an abstract class.
  - c) The base class function is called instead.
  - d) The function is removed from memory.
5. **Which keyword is used to declare a pure virtual function in C++?**
  - a) `final`
  - b) `abstract`
  - c) `= 0`
  - d) `override`

## 10.10 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective C++** – Scott Meyers
- **Object-Oriented Programming with C++** – E. Balagurusamy

# Chapter 11: Exception Handling in C++

## 11.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what exceptions are** and why they occur.
- Learn how to **handle runtime errors** using the **try-catch mechanism**.
- Explore **throwing exceptions** and **catching multiple exceptions**.
- Understand the use of **standard exception classes** in C++.

---

## 11.2 Introduction to Exception Handling

An **exception** is an unexpected event that **disrupts program execution**. C++ provides a mechanism to handle such errors using **exception handling**, which prevents **program crashes** and enables **graceful error recovery**.

- ✓ **Prevents program termination due to errors.**
- ✓ **Separates error-handling logic from normal program flow.**
- ✓ **Makes debugging easier and improves program reliability.**

---

## 11.3 The Try-Catch Mechanism in C++

### 11.3.1 Syntax of Exception Handling

C++ provides three keywords for handling exceptions:

- **try** – Defines a block where an exception might occur.
- **throw** – Throws an exception when an error is encountered.
- **catch** – Handles the thrown exception.

*Example: Basic Try-Catch Block*

```
#include <iostream>
using namespace std;

int main() {
    try {
        int num = 10;
        if (num > 5) {
            throw "Number is too large!";
        }
    }
}
```

```

    }
    cout << "Number: " << num << endl;
} catch (const char* msg) {
    cout << "Exception caught: " << msg << endl;
}
return 0;
}

```

#### □ Output:

Exception caught: Number is too large!

- ✓ The program **does not crash** but instead handles the error gracefully.

## 11.4 Throwing and Catching Exceptions

### 11.4.1 Throwing an Exception

- ✓ The `throw` keyword is used to **generate an exception**.
- ✓ The exception **must be caught** by a matching `catch` block.

#### *Example: Throwing an Exception*

```

#include <iostream>
using namespace std;

void checkAge(int age) {
    if (age < 18) {
        throw "Age must be 18 or above.";
    }
    cout << "Access granted!" << endl;
}

int main() {
    try {
        checkAge(16);
    } catch (const char* msg) {
        cout << "Error: " << msg << endl;
    }
    return 0;
}

```

#### □ Output:

Error: Age must be 18 or above.

- ✓ If `age < 18`, the function **throws an exception**, which is caught in `main()`.

## 11.5 Catching Multiple Exceptions

- ✓ **Multiple catch blocks** handle different types of exceptions.
- ✓ Each `catch` block must match the type of thrown exception.

*Example: Catching Different Exception Types*

```
#include <iostream>
using namespace std;

void divide(int a, int b) {
    if (b == 0) {
        throw 0; // Throwing an integer exception
    }
    cout << "Result: " << a / b << endl;
}

int main() {
    try {
        divide(10, 0);
    } catch (int error) {
        cout << "Error: Division by zero is not allowed!" << endl;
    }
    return 0;
}
```

□ **Output:**

Error: Division by zero is not allowed!

✓ **throw 0 matches catch (int error), handling division by zero safely.**

## 11.6 Using Standard Exception Classes in C++

✓ C++ provides built-in **exception classes** in the `<stdexcept>` library.

✓ These classes handle common errors like **invalid input, out-of-range values, and division by zero**.

**Example: Using Standard Exceptions**

```
#include <iostream>
#include <stdexcept> // Include standard exceptions
using namespace std;

void testFunction(int value) {
    if (value < 0) {
        throw invalid_argument("Negative value is not allowed");
    }
    cout << "Valid input: " << value << endl;
}

int main() {
    try {
        testFunction(-5);
    } catch (const invalid_argument& e) {
        cout << "Exception: " << e.what() << endl;
    }
    return 0;
}
```

□ **Output:**



Exception: Negative value is not allowed

✓ **invalid\_argument** is a predefined exception class in `<stdexcept>`.

## 11.7 Handling All Exceptions Using `catch(...)`

- ✓ The **catch-all handler** (`catch(...)`) catches **any type of exception**.
- ✓ It is useful when the exception type is unknown.

*Example: Catch-All Handler*

```
#include <iostream>
using namespace std;

int main() {
    try {
        throw 3.14; // Throwing a double exception
    } catch (...) {
        cout << "An exception was caught!" << endl;
    }
    return 0;
}
```

□ **Output:**

An exception was caught!

- ✓ The **catch(...)** block handles any thrown exception.

## 11.8 Summary

- ✓ **Exception handling** prevents program crashes by catching runtime errors.
- ✓ **try-catch blocks** handle exceptions and allow controlled execution flow.
- ✓ **Throwing exceptions** (`throw`) signals that an error has occurred.
- ✓ **Catching multiple exceptions** ensures proper handling of different error types.
- ✓ **Standard exception classes** simplify exception handling for common errors.
- ✓ **catch(...)** is a generic catch block that catches all exceptions.

## 11.9 Keywords

- **Exception Handling**
- **try-catch Mechanism**
- **Throwing Exceptions**
- **Standard Exception Classes**
- **Catch-All Handler** (`catch(...)`)

### 11.10 Multiple-Choice Questions (MCQs)

1. **What does an exception in C++ represent?**
    - a) A syntax error in the program
    - b) A runtime error that disrupts program execution
    - c) A special type of variable
    - d) A loop control statement
  2. **Which keyword is used to handle exceptions in C++?**
    - a) `error`
    - b) `try`
    - c) `handle`
    - d) `catch`
  3. **What will happen if an exception is thrown but not caught?**
    - a) The program will continue execution normally
    - b) The program will terminate abnormally
    - c) The exception will be ignored
    - d) The program will enter an infinite loop
  4. **Which standard library header contains built-in exception classes?**
    - a) `<iostream>`
    - b) `<stdexcept>`
    - c) `<fstream>`
    - d) `<math.h>`
  5. **Which catch block will catch any type of exception?**
    - a) `catch(int e)`
    - b) `catch(...)`
    - c) `catch(std::exception e)`
    - d) `catch(std::string e)`
- 

### 11.11 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective C++** – Scott Meyers
- **C++ Primer (5th Edition)** – Stanley B. Lippman

## Chapter 12: Standard Template Library (STL) in C++

### 12.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what the Standard Template Library (STL)** is and why it is useful.
- Learn about **STL containers (vector, list, set, map, etc.)** and their usage.
- Explore **STL algorithms** for sorting, searching, and modifying data.
- Understand **iterators** and how they help navigate STL containers.

---

## 12.2 Introduction to STL

The **Standard Template Library (STL)** in C++ is a collection of **ready-to-use** data structures and algorithms. It provides:

- ✓ **Predefined containers** like vectors, lists, and maps.
- ✓ **Built-in algorithms** like sorting and searching.
- ✓ **Iterators** to traverse data structures efficiently.

### 12.2.1 Why Use STL?

- ✓ **Reduces coding effort** – No need to implement data structures from scratch.
- ✓ **Optimized for performance** – Uses efficient memory management.
- ✓ **Generic programming** – Works with any data type via templates.

---

## 12.3 STL Components

STL is divided into **three major components**:

| Component         | Description                              | Examples                  |
|-------------------|------------------------------------------|---------------------------|
| <b>Containers</b> | Stores collections of objects.           | vector, list, map, set    |
| <b>Algorithms</b> | Performs operations on containers.       | sort(), find(), reverse() |
| <b>Iterators</b>  | Used to traverse elements in containers. | begin(), end(), advance() |

---

## 12.4 STL Containers

STL **containers** are classified into three types:

| Container Type                | Description                          | Examples                     |
|-------------------------------|--------------------------------------|------------------------------|
| <b>Sequence Containers</b>    | Stores data in linear order.         | vector, deque, list          |
| <b>Associative Containers</b> | Uses key-value pairs.                | set, map, multimap           |
| <b>Unordered Containers</b>   | Stores data in an unordered fashion. | unordered_map, unordered_set |

---

### 12.4.1 Vectors: Dynamic Arrays in C++

- ✓ **Similar to arrays but dynamically resizable.**
- ✓ **Allows random access to elements using indices.**

*Example: Using Vectors*

```
#include <iostream>
```

```
#include <vector>
using namespace std;

int main() {
    vector<int> numbers = {10, 20, 30, 40, 50};

    // Adding elements
    numbers.push_back(60);

    // Accessing elements
    cout << "First element: " << numbers[0] << endl;

    // Iterating through vector
    for (int num : numbers) {
        cout << num << " ";
    }

    return 0;
}
```

□ **Output:**

```
First element: 10
10 20 30 40 50 60
```

✓ **Vectors automatically resize when new elements are added.**

## 12.4.2 Maps: Key-Value Pair Storage

✓ **Stores elements as key-value pairs.**

✓ **Keys are unique and sorted.**

*Example: Using Maps*

```
#include <iostream>
#include <map>
using namespace std;

int main() {
    map<int, string> students;

    // Inserting values
    students[101] = "Alice";
    students[102] = "Bob";

    // Accessing elements
    cout << "Student 101: " << students[101] << endl;

    return 0;
}
```

□ **Output:**

```
Student 101: Alice
```

✓ **Maps store elements in ascending order of keys.**

---

## 12.5 STL Algorithms

### 12.5.1 Sorting Elements in a Vector

✓ The `sort()` function sorts elements in ascending order.

*Example: Sorting with `sort()`*

```
#include <iostream>
#include <vector>
#include <algorithm> // For sort()
using namespace std;

int main() {
    vector<int> nums = {5, 2, 8, 1, 3};

    sort(nums.begin(), nums.end()); // Sorting in ascending order

    for (int num : nums) {
        cout << num << " ";
    }

    return 0;
}
```

□ **Output:**

1 2 3 5 8

✓ Sorting is optimized using quicksort, heapsort, or mergesort.

---

## 12.6 STL Iterators

✓ Iterators allow traversing elements in STL containers.

✓ Useful when working with complex data structures like maps and sets.

*Example: Iterating Through a Vector Using Iterators*

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> nums = {10, 20, 30, 40};

    vector<int>::iterator it;
    for (it = nums.begin(); it != nums.end(); ++it) {
        cout << *it << " ";
    }

    return 0;
}
```

□ **Output:**

10 20 30 40

✓ Iterators provide a flexible way to access elements without using indices.

---

## 12.7 Summary

- ✓ STL provides predefined containers, algorithms, and iterators.
  - ✓ Vectors store dynamic arrays, maps store key-value pairs.
  - ✓ Algorithms like `sort()` and `find()` make operations efficient.
  - ✓ Iterators allow easy traversal of STL containers.
- 

## 12.8 Keywords

- STL (Standard Template Library)
  - Containers (Vector, List, Map, Set)
  - Algorithms (sort, find, reverse)
  - Iterators
  - Associative Containers
- 

## 12.9 Multiple-Choice Questions (MCQs)

1. Which STL container uses key-value pairs?
    - a) `vector`
    - b) `set`
    - c) `map`
    - d) `list`
  2. Which algorithm is used to sort elements in STL?
    - a) `search()`
    - b) `sort()`
    - c) `find()`
    - d) `push_back()`
  3. What does an STL iterator do?
    - a) Stores elements in a container
    - b) Helps traverse elements in a container
    - c) Deletes elements from a container
    - d) Finds the smallest element
  4. Which container stores elements in sequential order?
    - a) `vector`
    - b) `map`
    - c) `set`
    - d) `unordered_map`
  5. Which STL container allows duplicate keys?
    - a) `map`
    - b) `multimap`
    - c) `vector`
    - d) `unordered_set`
-

## 12.10 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective STL** – Scott Meyers
- **C++ Primer (5th Edition)** – Stanley B. Lippman

# Chapter 13: Smart Pointers in C++

## 13.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what smart pointers are** and why they are useful.
- Learn about **memory management issues** in C++ and how smart pointers solve them.
- Explore different types of **smart pointers** (`unique_ptr`, `shared_ptr`, `weak_ptr`).
- Implement **smart pointers** in real-world applications.

---

## 13.2 Introduction to Smart Pointers

A **smart pointer** is a C++ object that manages dynamically allocated memory. Unlike raw pointers (`new/delete`), smart pointers **automatically free memory** when it is no longer needed.

- ✓ **Prevents memory leaks** and dangling pointers.
- ✓ **Eliminates manual delete calls.**
- ✓ **Ensures safe and efficient memory management.**

### Example: Raw Pointer vs. Smart Pointer

```
#include <iostream>
#include <memory> // For smart pointers
using namespace std;

int main() {
    // Using raw pointer (Manual delete required)
    int* ptr = new int(10);
    cout << *ptr << endl;
    delete ptr; // Must delete manually

    // Using smart pointer (Auto delete)
    unique_ptr<int> smartPtr = make_unique<int>(20);
    cout << *smartPtr << endl; // No need for manual delete

    return 0;
}
```

## □ Output:

10  
20

✓ `unique_ptr` automatically deletes memory when it goes out of scope.

### 13.3 Why Use Smart Pointers?

| Problem with Raw Pointers                             | Solution with Smart Pointers  |
|-------------------------------------------------------|-------------------------------|
| Requires manual <code>delete</code> calls.            | Automatically deletes memory. |
| Causes memory leaks if <code>delete</code> is missed. | Prevents memory leaks.        |
| Leads to segmentation faults if used after deletion.  | Avoids dangling pointers.     |

### 13.4 Types of Smart Pointers in C++

#### 1 □ `unique_ptr` – Exclusive Ownership

- ✓ A `unique_ptr` owns an object exclusively (no sharing).
- ✓ When a `unique_ptr` goes out of scope, it **automatically deletes** the allocated object.
- ✓ **Cannot be copied**, only moved using `std::move()`.

*Example: Using `unique_ptr`*

```
#include <iostream>
#include <memory> // Include smart pointers
using namespace std;

class Car {
public:
    Car() { cout << "Car created" << endl; }
    ~Car() { cout << "Car destroyed" << endl; }
    void drive() { cout << "Car is driving" << endl; }
};

int main() {
    unique_ptr<Car> car1 = make_unique<Car>(); // Allocates memory
    car1->drive();

    // unique_ptr cannot be copied
    // unique_ptr<Car> car2 = car1; // □ ERROR

    unique_ptr<Car> car2 = move(car1); // □ Ownership transferred
    car2->drive();

    return 0;
}
```

## □ Output:

Car created  
Car is driving  
Car is driving  
Car destroyed

✓ **Memory is automatically freed** when `unique_ptr` goes out of scope.



---

## 2 `shared_ptr` – Shared Ownership

- ✓ Multiple `shared_ptr` instances can **share ownership** of the same object.
- ✓ **Reference count tracks ownership** and deletes the object when the count reaches zero.

*Example: Using `shared_ptr`*

```
cpp
CopyEdit
#include <iostream>
#include <memory>
using namespace std;

class Car {
public:
    Car() { cout << "Car created" << endl; }
    ~Car() { cout << "Car destroyed" << endl; }
    void drive() { cout << "Car is driving" << endl; }
};

int main() {
    shared_ptr<Car> car1 = make_shared<Car>(); // Shared ownership
    shared_ptr<Car> car2 = car1; // car2 also owns the object

    cout << "Reference count: " << car1.use_count() << endl; // 2 owners

    return 0;
}
```

### **Output:**

```
Car created
Reference count: 2
Car destroyed
```

- ✓ `shared_ptr` **deletes the object only when the last reference goes out of scope.**
- 

## 3 `weak_ptr` – Avoiding Circular References

- ✓ `weak_ptr` **does not increase the reference count.**
- ✓ Prevents **circular dependencies** between `shared_ptr` objects.

*Example: Using `weak_ptr`*

```
#include <iostream>
#include <memory>
using namespace std;

class Car {
public:
    Car() { cout << "Car created" << endl; }
    ~Car() { cout << "Car destroyed" << endl; }
    void drive() { cout << "Car is driving" << endl; }
};
```

```
int main() {
    shared_ptr<Car> car1 = make_shared<Car>();
    weak_ptr<Car> car2 = car1; // Does not increase reference count

    cout << "Reference count: " << car1.use_count() << endl;

    return 0;
}
```

#### □ Output:

```
Car created
Reference count: 1
Car destroyed
```

✓ **weak\_ptr** prevents circular dependencies between objects.

---

## 13.5 Choosing the Right Smart Pointer

| Smart Pointer | Use Case                                                                               |
|---------------|----------------------------------------------------------------------------------------|
| unique_ptr    | When <b>only one owner</b> should manage the object.                                   |
| shared_ptr    | When <b>multiple owners</b> share the same object.                                     |
| weak_ptr      | When an object <b>should not increase reference count</b> (avoid circular dependency). |

---

## 13.6 Summary

- ✓ **Smart Pointers automatically manage memory**, preventing leaks and dangling pointers.
- ✓ **unique\_ptr provides exclusive ownership** and cannot be copied.
- ✓ **shared\_ptr enables multiple ownership**, deleting memory when the last reference is gone.
- ✓ **weak\_ptr prevents circular dependencies** between shared pointers.

---

## 13.7 Keywords

- **Smart Pointers**
- **Memory Management**
- **unique\_ptr, shared\_ptr, weak\_ptr**
- **Reference Counting**
- **Automatic Memory Deallocation**

---

## 13.8 Multiple-Choice Questions (MCQs)

1. **Which smart pointer ensures exclusive ownership of an object?**
  - a) shared\_ptr
  - b) unique\_ptr

- c) `weak_ptr`
- d) `auto_ptr`
- 2. **What happens when the last `shared_ptr` to an object goes out of scope?**
  - a) The object is destroyed
  - b) The object remains in memory
  - c) The program crashes
  - d) It gets converted to a `weak_ptr`
- 3. **Which smart pointer is used to prevent circular dependencies?**
  - a) `unique_ptr`
  - b) `shared_ptr`
  - c) `weak_ptr`
  - d) `auto_ptr`
- 4. **Which header file must be included to use smart pointers?**
  - a) `<iostream>`
  - b) `<memory>`
  - c) `<vector>`
  - d) `<smartptr>`
- 5. **Which function is used to allocate memory for `unique_ptr`?**
  - a) `new`
  - b) `malloc()`
  - c) `make_unique()`
  - d) `create_ptr()`

---

### 13.9 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective Modern C++** – Scott Meyers
- **C++ Primer (5th Edition)** – Stanley B. Lippman

## Chapter 14: Multithreading and Concurrency in C++

### 14.1 Objectives

By the end of this chapter, you will be able to:

- Understand **what multithreading is** and its advantages.
- Learn how to **create and manage multiple threads** in C++.
- Explore **thread synchronization techniques** such as mutex and condition variables.
- Understand **race conditions** and how to prevent them.

---

### 14.2 Introduction to Multithreading

**Multithreading** allows a program to **execute multiple tasks simultaneously**, improving performance and responsiveness. Instead of running sequentially, multiple threads run in parallel.

- ✓ **Increases performance** by utilizing multi-core processors.
- ✓ **Improves responsiveness** in applications (e.g., GUI, real-time systems).
- ✓ **Used in parallel computing, gaming, and real-time processing.**

### 14.3 Creating Threads in C++

- ✓ **C++ provides the `<thread>` library** to create and manage threads.
- ✓ **A thread executes a function concurrently with the main program.**

#### *Example: Creating a Simple Thread*

```
#include <iostream>
#include <thread> // Include thread library
using namespace std;

void printMessage() {
    cout << "Hello from a separate thread!" << endl;
}

int main() {
    thread t1(printMessage); // Create a new thread
    t1.join(); // Wait for thread to finish
    cout << "Main function executed!" << endl;
    return 0;
}
```

#### □ **Output:**

```
Hello from a separate thread!
Main function executed!
```

- ✓ **The main thread waits for `t1` to finish** using `join()`.

### 14.4 Joining and Detaching Threads

- ✓ **`join()` waits for a thread to complete** before continuing execution.
- ✓ **`detach()` allows a thread to run independently.**

#### *Example: Using `detach()`*

```
#include <iostream>
#include <thread>
using namespace std;

void independentTask() {
    cout << "Thread is running independently" << endl;
}

int main() {
    thread t(independentTask);
    t.detach(); // The thread runs separately

    cout << "Main function finished" << endl;
}
```

```
    return 0;
}
```

#### □ Output:

```
Main function finished
Thread is running independently
```

✓ The thread **continues execution even after `main()` exits**.

## 14.5 Race Conditions and Synchronization

✓ **Race condition occurs** when multiple threads access a shared resource **simultaneously**, causing unpredictable results.

✓ **Synchronization tools** prevent race conditions.

### 14.5.1 Using Mutex for Synchronization

✓ **A mutex (mutual exclusion) locks shared resources** to prevent concurrent access.

*Example: Using `mutex` to Prevent Race Condition*

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

mutex mtx; // Declare a mutex

void printMessage(string msg) {
    mtx.lock(); // Lock before accessing shared resource
    cout << msg << endl;
    mtx.unlock(); // Unlock after execution
}

int main() {
    thread t1(printMessage, "Thread 1");
    thread t2(printMessage, "Thread 2");

    t1.join();
    t2.join();

    return 0;
}
```

#### □ Output (order may vary):

```
Thread 1
Thread 2
```

✓ **Mutex prevents threads from printing simultaneously, avoiding data corruption.**

## 14.5.2 Using Lock Guard for Safer Mutex Handling

✓ **lock\_guard automatically locks and unlocks a mutex.**

*Example: Using lock\_guard*

```
#include <iostream>
#include <thread>
#include <mutex>
using namespace std;

mutex mtx;

void safeFunction(string msg) {
    lock_guard<mutex> lock(mtx); // Auto-locks and unlocks
    cout << msg << endl;
}

int main() {
    thread t1(safeFunction, "Thread 1");
    thread t2(safeFunction, "Thread 2");

    t1.join();
    t2.join();

    return 0;
}
```

□ **Output:**

```
Thread 1
Thread 2
```

✓ **No need for manual lock() and unlock() calls.**

## 14.6 Condition Variables for Thread Communication

✓ **A condition variable allows threads to wait for a signal before continuing execution.**

✓ **Useful in producer-consumer problems** (where one thread produces data, and another consumes it).

*Example: Using Condition Variables*

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
using namespace std;

mutex mtx;
condition_variable cv;
bool ready = false;

void worker() {
    unique_lock<mutex> lock(mtx);
```

```

        cv.wait(lock, [] { return ready; }); // Wait for signal
        cout << "Worker thread executed!" << endl;
    }

int main() {
    thread t(worker);

    // Simulate some work
    this_thread::sleep_for(chrono::seconds(1));

    // Notify worker thread
    {
        lock_guard<mutex> lock(mtx);
        ready = true;
    }
    cv.notify_one();

    t.join();
    return 0;
}

```

#### □ Output:

Worker thread executed!

✓ **The worker thread waits until `ready` becomes `true` before running.**

## 14.7 Summary

- ✓ **Multithreading improves performance by running tasks in parallel.**
- ✓ **Threads can be joined (`join()`) or detached (`detach()`).**
- ✓ **Race conditions occur when multiple threads access shared resources.**
- ✓ **Mutex and lock guards ensure safe thread synchronization.**
- ✓ **Condition variables allow thread communication.**

## 14.8 Keywords

- **Multithreading**
- **Thread Synchronization**
- **Mutex (`mutex`)**
- **Race Condition**
- **Condition Variable (`condition_variable`)**

## 14.9 Multiple-Choice Questions (MCQs)

1. **What is the main benefit of multithreading?**
  - a) Reduces memory usage
  - b) Runs tasks in parallel for better performance
  - c) Eliminates the need for loops
  - d) Simplifies debugging

2. **What function ensures a thread completes before continuing execution?**
    - a) `detach()`
    - b) `join()`
    - c) `sleep()`
    - d) `notify()`
  3. **What is a race condition?**
    - a) When multiple threads modify the same resource unpredictably
    - b) A thread waiting indefinitely
    - c) A function executing without parameters
    - d) A way to speed up execution
  4. **Which of the following prevents race conditions?**
    - a) `mutex`
    - b) `condition_variable`
    - c) `detach()`
    - d) `sleep_for()`
  5. **Which keyword is required for thread creation in C++?**
    - a) `thread`
    - b) `task`
    - c) `worker`
    - d) `async`
- 

### 14.10 Further Reading

- **C++ Concurrency in Action** – Anthony Williams
- **The C++ Programming Language** – Bjarne Stroustrup
- **Effective Modern C++** – Scott Meyers

## Chapter 15: File Handling in C++

### 15.1 Objectives

By the end of this chapter, you will be able to:

- Understand **the concept of file handling** and why it is essential.
- Learn about **file streams** (`ifstream`, `ofstream`, `fstream`) in C++.
- Perform **reading and writing operations** on text and binary files.
- Understand **error handling in file operations**.

---

### 15.2 Introduction to File Handling

**File handling** in C++ allows reading from and writing to **external files** instead of working with standard input/output (`cin/cout`). This is useful for:

- ✓ **Storing and retrieving data** permanently.
- ✓ **Handling large datasets** (logs, configurations, databases).
- ✓ **Interacting with external files** (CSV, text, binary files).



C++ provides the **fstream library** to work with files, which includes:

| File Stream | Purpose                     |
|-------------|-----------------------------|
| ifstream    | Reads from a file.          |
| ofstream    | Writes to a file.           |
| fstream     | Reads and writes to a file. |

---

## 15.3 Opening and Closing Files

### 15.3.1 Opening a File

A file must be **opened** before performing any operations on it.

*Example: Opening a File for Writing*

```
#include <iostream>
#include <fstream> // Include file handling library
using namespace std;

int main() {
    ofstream file("example.txt"); // Open file for writing

    if (file.is_open()) {
        cout << "File opened successfully!" << endl;
    } else {
        cout << "Error opening file!" << endl;
    }

    file.close(); // Close the file
    return 0;
}
```

#### □ Output:

File opened successfully!

✓ The `ofstream` object **opens a file for writing** and then closes it.

---

## 15.4 Writing to a File

✓ Use `ofstream` to write data to a text file.

✓ The `<<` operator is used to insert data into the file.

*Example: Writing to a File*

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream file("data.txt"); // Create and open file

    if (file.is_open()) {
        file << "Hello, this is a test file!" << endl;
        file << "C++ File Handling Example" << endl;
    }
}
```

```

        cout << "Data written successfully!" << endl;
    }

    file.close();
    return 0;
}

```

#### □ Output:

Data written successfully!

✓ **A new file (data.txt) is created, and text is written into it.**

## 15.5 Reading from a File

✓ **Use `ifstream` to read data** from a file.

✓ **The `>>` operator** extracts data from the file.

#### *Example: Reading a File*

```

#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("data.txt"); // Open file for reading
    string line;

    if (file.is_open()) {
        while (getline(file, line)) { // Read line by line
            cout << line << endl;
        }
    } else {
        cout << "File could not be opened!" << endl;
    }

    file.close();
    return 0;
}

```

#### □ Output (from data.txt):

Hello, this is a test file!  
C++ File Handling Example

✓ **`getline()` reads text from the file line by line.**

## 15.6 Appending to a File

✓ **Use `ios::app` mode to add new content** to an existing file.

✓ **Data is *not* overwritten** when using append mode.

*Example: Appending Data to a File*

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ofstream file("data.txt", ios::app); // Open file in append mode

    if (file.is_open()) {
        file << "This is appended text!" << endl;
        cout << "Data appended successfully!" << endl;
    }

    file.close();
    return 0;
}
```

□ **Output:**

Data appended successfully!

✓ **ios::app ensures new content is added instead of overwriting the file.**

## 15.7 Working with Binary Files

- ✓ **Binary files store data in raw format** (not human-readable).
- ✓ **Use write() and read() functions** to handle binary data.

*Example: Writing to a Binary File*

```
#include <iostream>
#include <fstream>
using namespace std;

class Student {
public:
    char name[20];
    int age;

    void getData() {
        cout << "Enter name and age: ";
        cin >> name >> age;
    }
};

int main() {
    Student s;
    s.getData();

    ofstream file("student.dat", ios::binary);
    file.write((char*)&s, sizeof(s));
    file.close();

    cout << "Data written in binary file!" << endl;
    return 0;
}
```

```
}
```

#### □ **Output:**

```
Enter name and age: Alex 21
Data written in binary file!
```

✓ **Data is stored in raw binary format inside `student.dat`.**

#### *Example: Reading from a Binary File*

```
#include <iostream>
#include <fstream>
using namespace std;

class Student {
public:
    char name[20];
    int age;

    void showData() {
        cout << "Name: " << name << ", Age: " << age << endl;
    }
};

int main() {
    Student s;

    ifstream file("student.dat", ios::binary);
    file.read((char*)&s, sizeof(s));
    file.close();

    s.showData();
    return 0;
}
```

#### □ **Output (if data exists in `student.dat`):**

```
Name: Alex, Age: 21
```

✓ **Binary files are faster than text files for large data storage.**

## 15.8 Checking File Status (Error Handling)

✓ **Use `fail()` to check for file opening errors.**

#### *Example: Checking for Errors in File Handling*

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    ifstream file("nonexistent.txt"); // File does not exist
```

```

    if (file.fail()) {
        cout << "Error: File could not be opened!" << endl;
    }

    return 0;
}

```

#### □ Output:

Error: File could not be opened!

✓ This ensures the program does not crash if the file is missing.

## 15.9 Summary

- ✓ File handling allows reading/writing to external files in C++.
- ✓ `ofstream` writes to files, `ifstream` reads from files, `fstream` handles both.
- ✓ Binary files store data more efficiently than text files.
- ✓ Appending prevents overwriting data.
- ✓ Error handling prevents file operation failures.

## 15.10 Keywords

- File Handling
- `ifstream`, `ofstream`, `fstream`
- Binary Files
- Append Mode (`ios::app`)
- Error Handling (`fail()`)

## 15.11 Multiple-Choice Questions (MCQs)

1. Which file stream is used for reading a file?
  - a) `ofstream`
  - b) `ifstream`
  - c) `fstream`
  - d) `openfile`
2. Which mode allows appending to a file without overwriting existing data?
  - a) `ios::trunc`
  - b) `ios::app`
  - c) `ios::out`
  - d) `ios::binary`
3. Which function reads an entire line from a file?
  - a) `getline()`
  - b) `readline()`
  - c) `fetchline()`
  - d) `scanline()`

4. **Which file format is faster for large data storage?**
  - a) Text file
  - b) Binary file
  - c) CSV file
  - d) JSON file
5. **What does `fail()` function check in file handling?**
  - a) If the file has data
  - b) If the file failed to open
  - c) If the file is locked
  - d) If the file is empty

---

### 15.12 Further Reading

- **The C++ Programming Language** – Bjarne Stroustrup
- **C++ Primer (5th Edition)** – Stanley B. Lippman
- **Effective C++** – Scott Meyers

## Chapter 16: Best Practices and Optimization in C++

### 16.1 Objectives

By the end of this chapter, you will be able to:

- Understand **best practices for writing clean and efficient C++ code**.
- Learn about **code optimization techniques** for better performance.
- Explore **memory management strategies** to avoid memory leaks.
- Apply **efficient algorithms and data structures** for faster execution.

---

### 16.2 Writing Clean and Readable Code

- ✓ **Use meaningful variable and function names.**
- ✓ **Follow consistent indentation and spacing.**
- ✓ **Use comments and documentation where necessary.**
- ✓ **Keep functions short and focused on a single task.**

*Example: Good vs. Bad Coding Style*

#### ❑ **Bad Code (Hard to Read)**

```
#include <iostream>
using namespace std; int main() { int x=10,y=20;cout<<x+y;return 0;}
```

### ✓ Good Code (Readable & Well-Structured)

```
#include <iostream>
using namespace std;

int main() {
    int num1 = 10, num2 = 20;
    int sum = num1 + num2;
    cout << "Sum: " << sum << endl;
    return 0;
}
```

#### □ Output:

Sum: 30

### ✓ Proper formatting improves readability and maintainability.

## 16.3 Memory Management Best Practices

- ✓ Avoid memory leaks by always freeing dynamically allocated memory.
- ✓ Use smart pointers (`unique_ptr`, `shared_ptr`) instead of raw pointers.
- ✓ Minimize unnecessary memory allocations.

#### *Example: Memory Leak (Bad Practice)*

```
void func() {
    int* ptr = new int(10);
    // Memory is allocated but never freed
}
```

### ✓ Fixed using Smart Pointers

```
#include <iostream>
#include <memory>
using namespace std;

void func() {
    unique_ptr<int> ptr = make_unique<int>(10); // Auto memory management
}
```

### ✓ Smart pointers ensure memory is freed automatically.

## 16.4 Optimizing Loops for Performance

- ✓ Use `++i` instead of `i++` in loops to avoid unnecessary copy operations.
- ✓ Use `reserve()` for vectors to prevent frequent memory reallocation.
- ✓ Minimize operations inside loops to improve efficiency.

#### *Example: Optimized vs. Non-Optimized Loops*

#### □ Unoptimized Loop

```
vector<int> nums;
for (int i = 0; i < 100000; i++) {
    nums.push_back(i); // Causes multiple memory reallocations
}
```

### ✓ Optimized Loop using `reserve()`

```
vector<int> nums;
nums.reserve(100000); // Pre-allocate memory
for (int i = 0; i < 100000; i++) {
    nums.push_back(i); // No reallocation needed
}
```

### ✓ Avoids multiple memory reallocations, making execution faster.

---

## 16.5 Using Efficient Data Structures

- ✓ Use appropriate data structures to optimize performance.
- ✓ Prefer `unordered_map` over `map` when order is not required.
- ✓ Use `vector` instead of `list` when frequent insertions/deletions are not needed.

| Use Case                      | Best Data Structure        |
|-------------------------------|----------------------------|
| Fast lookups                  | <code>unordered_map</code> |
| Sorted data storage           | <code>map</code>           |
| Dynamic arrays                | <code>vector</code>        |
| Frequent insertions/deletions | <code>list</code>          |

### *Example: Using `unordered_map` for Faster Lookups*

```
#include <iostream>
#include <unordered_map>
using namespace std;

int main() {
    unordered_map<int, string> students;
    students[1] = "Alice";
    students[2] = "Bob";

    cout << students[1] << endl; // Fast lookup
    return 0;
}
```

### ✓ `unordered_map` provides average $O(1)$ lookup time instead of $O(\log n)$ in `map`.

---

## 16.6 Reducing Function Overhead with Inline Functions

- ✓ **Inline functions** reduce function call overhead by replacing function calls with actual code.

### *Example: Using `inline` for Performance*

```
#include <iostream>
using namespace std;
```



```

inline int add(int a, int b) {
    return a + b;
}

int main() {
    cout << add(10, 20) << endl; // Faster execution
    return 0;
}

```

✓ **Inline functions improve execution speed for small, frequently used functions.**

## 16.7 Using Move Semantics for Performance

✓ **Avoid unnecessary copying of large objects using `move()`.**

✓ **Use `std::move()` to transfer ownership instead of making deep copies.**

*Example: Optimizing Object Transfers Using `move()`*

```

#include <iostream>
#include <vector>
using namespace std;

vector<int> getVector() {
    vector<int> temp = {1, 2, 3, 4, 5};
    return move(temp); // Avoids deep copy
}

int main() {
    vector<int> numbers = getVector();
    cout << "Vector received!" << endl;
    return 0;
}

```

✓ **Move semantics prevent expensive deep copies of large objects.**

## 16.8 Summary

✓ **Clean and readable code** improves maintainability.

✓ **Proper memory management** prevents leaks and crashes.

✓ **Loop optimizations** improve performance.

✓ **Choosing the right data structures** makes operations faster.

✓ **Move semantics and inline functions** reduce execution overhead.

## 16.9 Keywords

- **Memory Management**
- **Loop Optimization**
- **Data Structures Efficiency**
- **Move Semantics (`std::move`)**

- **Function Inlining**

---

### 16.10 Multiple-Choice Questions (MCQs)

1. **Which data structure is best for fast lookups?**
  - a) `list`
  - b) `vector`
  - c) `map`
  - d) `unordered_map`
2. **Which function is used to prevent unnecessary deep copying of objects?**
  - a) `copy()`
  - b) `move()`
  - c) `duplicate()`
  - d) `clone()`
3. **Why should we use `reserve()` with vectors?**
  - a) To allocate exact memory needed.
  - b) To reduce the number of reallocations.
  - c) To make the vector immutable.
  - d) To allow multiple data types.
4. **Which feature helps reduce function call overhead?**
  - a) Recursion
  - b) Virtual functions
  - c) Inline functions
  - d) Friend functions
5. **Which of the following is a major cause of memory leaks?**
  - a) Using smart pointers
  - b) Forgetting to delete dynamically allocated memory
  - c) Using references
  - d) Declaring large arrays

---

### 16.11 Further Reading

- **Effective C++** – Scott Meyers
- **The C++ Programming Language** – Bjarne Stroustrup
- **C++ Primer (5th Edition)** – Stanley B. Lippman